Как Go запускает сам себя

От точки входа ОС до первой строки `main.main`.

```
rt0 → m0/g0 → schedinit
procresize → runtime.main
init → main.main
```

Знакомые G/M/P пока не существуют

ЗАДАЧА

В углубленной лекции про рантайм мы изучили готовую машину: G, M, P, очереди выполнения и `schedule`.

МЕХАНИЗМ

После загрузчика ОС существует только первый поток ОС, исходный стек и отображенные данные. Кто создал планировщик?

ИНЖЕНЕРНЫЙ ВЫВОД

Задача 4.6 — восстановить порядок появления уже знакомых механизмов.

Вся последовательность запуска

ПОСЛЕДОВАТЕЛЬНОСТЬ

```
Загрузчик ОС
↓ точка входа _rt0_*
runtime.rt0_go
↓ статические m0/g0 + TLS
osinit → schedinit
↓ создание набора P
newproc(runtime.main) → mstart
↓ планировщик выбирает G
runtime.main → init → main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Главная идея: рантайм создает собственную рабочую среду поэтапно.

Обычный код хочет уже работающий рантайм

ЗАДАЧА

Нужны горутины, аллокации, локи, карты и текущая G. Но кто выполнит код, который их создает?

МЕХАНИЗМ

Рантайм начинает со статических данных, первого потока ОС и его исходного стека.

ИНЖЕНЕРНЫЙ ВЫВОД

Первые ресурсы приходят от ОС и компоновщика.

Что ОС уже создала к точке входа?

ЗАДАЧА И КОНТЕКСТ

Загрузчик ОС закончил работу. Есть виртуальное адресное пространство, первый поток, стек, регистры и отображенные данные исполняемого файла.

МЕХАНИЗМ

```
код и статические данные уже отображены
↓ первый поток ОС и его стек
argc / argv / env / auxv
↓ управление по точке входа
```

ИНЖЕНЕРНЫЙ ВЫВОД

Go еще не создал ни планировщик, ни обычные горутины.

Чего пока нет в Go-процессе?

ЗАДАЧА И КОНТЕКСТ

К моменту входа в программу еще нет ни P, ни локальных очередей, ни обычных G, ни фоновых служб рантайма.

МЕХАНИЗМ

```
есть: поток ОС + исходный стек
+ отображенные статические данные

нет: P / очередей / рабочих G
нет: sysmon / GC workers
```

ИНЖЕНЕРНЫЙ ВЫВОД

Первый Go-код должен построить эти механизмы из доступных ресурсов ОС.

Первый переход: `_rt0_amd64` `rt0_go`

ОПОРНАЯ МОДЕЛЬ

Точка входа подготавливает `argc/argv` и передает управление общему начальному коду рантайма.

АЛГОРИТМ

```
ОС → _rt0_amd64
↓ передать argc/argv
runtime.rt0_go
```

ИНЖЕНЕРНЫЙ ВЫВОД

Загрузчик ОС не вызывает `main.main` напрямую.

Почему `_rt0_amd64` почти ничего не делает

ЗАДАЧА И КОНТЕКСТ

`_rt0_amd64` — небольшая точка входа, которая адаптирует соглашение ОС к общему стартовому коду Go.

МЕХАНИЗМ

```
ОС → _rt0_amd64
↓ подготовка argc/argv
runtime.rt0_go
```

ИНЖЕНЕРНЫЙ ВЫВОД

На другой архитектуре меняется входной код, но цель перехода та же.

Внутренняя и внешняя компоновка: разные входы

ЗАДАЧА И КОНТЕКСТ

При внутренней и внешней компоновке внешний путь входа может отличаться: либо `_rt0_amd64`, либо стартовый код платформы.

МЕХАНИЗМ

```
внутренняя компоновка
→ _rt0_amd64
внешняя компоновка
→ стартовый символ main платформы
оба пути → runtime.rt0_go
```

ИНЖЕНЕРНЫЙ ВЫВОД

Разные форматы запуска сходятся на общей первичной инициализации Go.

Почему стартовый код написан на ассемблере

ЗАДАЧА И КОНТЕКСТ

Обычная Go-функция предполагает уже настроенные G/M, стек и соглашения рантайма. На самой ранней стадии этих условий нет.

МЕХАНИЗМ

```
сначала: ассемблер rt0_go
↓ подготовить G/M и стек
↓ настроить доступ к текущей G
затем: обычные вызовы рантайма
```

ИНЖЕНЕРНЫЙ ВЫВОД

Первый этап приходится писать без предположений о готовом Go-рантайме.

Первый вопрос `rt0_go`: где стек?

ЗАДАЧА И КОНТЕКСТ

ОС дала начальному потоку стек, но рантайм еще не записал его границы в структуру G.

МЕХАНИЗМ

```
первый стек потока ОС
↓ подготовить границы
g0.stack.lo / g0.stack.hi
↓ настроить защитные границы
```

ИНЖЕНЕРНЫЙ ВЫВОД

Для запуска `g0` используется уже существующий системный стек.

Появляется g0

ЗАДАЧА И КОНТЕКСТ

g0 — служебная G потока M: на ней выполняются системные и планировочные функции.

МЕХАНИЗМ

```
обычная G → стек выделяет рантайм
но аллокатор пока не работает
↓
g0 использует стек первого потока ОС
```

ИНЖЕНЕРНЫЙ ВЫВОД

Первую служебную G нельзя создать обычным путем.

m0 и g0 существуют до аллокатора

GO / КОМАНДА

```
var (
    m0 m
    g0 g
)
```

ЧТО ПРОИСХОДИТ

Их память уже отображена загрузчиком ОС вместе с данными программы. Обычный new пока недоступен.

ИНЖЕНЕРНЫЙ ВЫВОД

Это статические объекты, а не результат вызова newm.

Первый прием: статические m0 и g0

ЗАДАЧА И КОНТЕКСТ

Обычные M/G появляются через работающие механизмы рантайма. Для самых первых объектов это невозможно.

МЕХАНИЗМ

```
m0 и g0 — статические данные
↓ ОС уже отобразила их память
↓ привязать к первому потоку
потом рантайм создаст остальные M/G
```

ИНЖЕНЕРНЫЙ ВЫВОД

Статическое размещение разрывает циклическую зависимость запуска.

m0: ОС уже создала первый поток

ЗАДАЧА И КОНТЕКСТ

m0 представляет первый поток ОС, который уже существует после загрузки программы.

МЕХАНИЗМ

```
первый поток ОС
↓ связать с объектом m0

не нужен вызов clone / newosproc
для создания этого первого потока
```

ИНЖЕНЕРНЫЙ ВЫВОД

Первый M получает поток от ОС, а не создает его самостоятельно.

g0: используем исходный стек ОС

ЗАДАЧА И КОНТЕКСТ

g0 — служебная G для m0. Ее стек не выделяется обычным аллокатором горутин.

МЕХАНИЗМ

```
стек первого потока ОС
↓ использовать для g0
↓ записать границы и защиту
готов системный стек
```

ИНЖЕНЕРНЫЙ ВЫВОД

На старте g0 использует уже существующий стек ОС.

`rt0_go` делает исходный стек стеком `g0`

ОПОРНАЯ МОДЕЛЬ

Исходный указатель SP уже указывает в стек, подготовленный ОС. Нужно заполнить границы стека `g0`.

АЛГОРИТМ

```
SP → начало и верх стека
↓ g0.stack.lo/hi
↓ stackguard0/stackguard1
↓ вызовы кода рантайма
```

ИНЖЕНЕРНЫЙ ВЫВОД

Не выделяем стек заново — используем уже существующий.

Почему начальные границы стека могут быть неточными

ЗАДАЧА И КОНТЕКСТ

На раннем этапе рантайм еще не знает все платформенные детали размера стека. Поэтому сначала задаются рабочие начальные границы.

МЕХАНИЗМ

```
первоначальные границы стека
↓ продолжить безопасный запуск
minit / платформенная настройка
↓ при необходимости уточнить границы
```

ИНЖЕНЕРНЫЙ ВЫВОД

Ранние границы нужны до завершения полной настройки потока.

Почему g0 нужен прямо сейчас

ЗАДАЧА И КОНТЕКСТ

`runtime.args`, `runtime.osinit` и `runtime.schedinit` уже выполняют код рантайма Go.

МЕХАНИЗМ

```
вызовы рантайма требуют текущую G
↓
первый поток ОС связан с g0
↓ getg() возвращает g0
```

ИНЖЕНЕРНЫЙ ВЫВОД

До вызова обычных функций рантайм должен установить свою первую G.

Почему одной глобальной g0 недостаточно

ЗАДАЧА И КОНТЕКСТ

Позднее несколько потоков ОС одновременно будут исполнять разные G. Одна глобальная переменная для текущей G не подойдет.

МЕХАНИЗМ

```
M1 → G17
M2 → G42
↓
каждый поток ОС хранит
собственную текущую G
```

ИНЖЕНЕРНЫЙ ВЫВОД

Связь текущей G с потоком должна быть локальной для этого потока.

getg()

ЗАДАЧА И КОНТЕКСТ

`getg()` позволяет рантайму быстро найти G, выполняемую текущим потоком ОС.

МЕХАНИЗМ

```
текущий поток ОС
↓ механизм TLS / регистр
getg()
↓ текущая G
```

ИНЖЕНЕРНЫЙ ВЫВОД

Механизм получения текущей G должен заработать до планировщика.

Почему текущая G привязана к потоку ОС

ЗАДАЧА И КОНТЕКСТ

Два потока ОС могут параллельно выполнять G17 и G42. Запись в общую `currentG` приводила бы к конфликту.

МЕХАНИЗМ

```
M1 → G17
M2 → G42

поэтому: поток ОС
→ собственное значение текущей G
```

ИНЖЕНЕРНЫЙ ВЫВОД

Глобальное состояние не может заменить локальную привязку G к потоку.

TLS и регистр текущей G

ЗАДАЧА И КОНТЕКСТ

Механизм текущей G зависит от архитектуры и платформы. На amd64 задействованы TLS и специальные соглашения о регистрах.

МЕХАНИЗМ

```
TLS: значение, привязанное к потоку
+ регистр g в ABIInternal
↓
быстрый доступ к текущей G
```

ИНЖЕНЕРНЫЙ ВЫВОД

Точная реализация различается между платформами; контракт один.

Почему модель «G всегда в TLS» неполна

ЗАДАЧА И КОНТЕКСТ

Модель «текущая G всегда лежит в TLS» неполна: сгенерированный Go-код может держать g в специальном регистре.

МЕХАНИЗМ

```
TLS ↔ регистр g
↓ переходы ABI / cgo
↓ восстановление состояния
getg() получает текущую G
```

ИНЖЕНЕРНЫЙ ВЫВОД

TLS и выделенные регистры совместно реализуют доступ к G.

`rt0_go` настраивает TLS

ЗАДАЧА И КОНТЕКСТ

`rt0_go` сначала настраивает TLS, проверяет его работу и записывает ссылку на `g0`.

МЕХАНИЗМ

```
настройка TLS
↓ проверка чтения/записи
↓ сохранить указатель g0
↓ обычные вызовы рантайма
```

ИНЖЕНЕРНЫЙ ВЫВОД

Если связь TLS и G ошибочна, дальнейшая работа рантайма некорректна.

Связь m0 ↔ g0 задает первый runtime-контекст

ОПОРНАЯ МОДЕЛЬ

После настройки current-G каждый поток ОС должен иметь свою служебную G и M.

АЛГОРИТМ

```
m0.g0 = &g0
g0.m  = &m0
↓
getg() → g0 → g0.m → m0
```

ИНЖЕНЕРНЫЙ ВЫВОД

Теперь первый поток ОС понимается в терминах рантайма.

От потока ОС к сущностям рантайма

ЗАДАЧА И КОНТЕКСТ

До связи `m0/g0` рантайм видит только поток ОС. После связи у него появляется минимальный собственный контекст выполнения.

МЕХАНИЗМ

```
первый поток ОС
↓ m0.g0 = &g0
↓ g0.m = &m0
getg() → g0 → m0
```

ИНЖЕНЕРНЫЙ ВЫВОД

С этого момента первый поток описан через сущности рантайма Go.

Но m0 еще не полноценный обычный M

ЗАДАЧА И КОНТЕКСТ

Связь `m0 ↔ g0` установлена, но остальные структуры рантайма еще отсутствуют.

МЕХАНИЗМ

```
пока нет: P, очередей выполнения
аллокатора и состояния GC
общих списков M / обработки сигналов
↓ schedinit подготовит эти механизмы
```

ИНЖЕНЕРНЫЙ ВЫВОД

Наличие первого M не равно запуску полноценного планировщика.

Необязательная ветка: cgo при запуске

ЗАДАЧА И КОНТЕКСТ

При cgo ранняя инициализация может вызвать `_cgo_init`, чтобы согласовать TLS, стек и состояние потоков ОС.

МЕХАНИЗМ

```
rt0_go
↓ при наличии cgo: _cgo_init
↓ настройка TLS / стека
↓ пересчитать защитные границы
```

ИНЖЕНЕРНЫЙ ВЫВОД

cgo добавляет платформенную ветку до запуска обычного Go-кода.

Почему начальный путь с cgo сложнее

ЗАДАЧА И КОНТЕКСТ

При cgo рантайм Go должен согласовать свою модель потоков со средой C еще до работы планировщика.

МЕХАНИЗМ

```
рантайм Go ↔ среда потоков C
TLS / границы стека / сигналы
↓
дальнейший общий путь rt0_go
```

ИНЖЕНЕРНЫЙ ВЫВОД

Основная схема запуска проще для программы без cgo.

Проверка поддерживаемых инструкций CPU

ПРОБЛЕМА И КОНТЕКСТ

МЕХАНИЗМ

```
GOAMD64=v2/v3/v4
↓
CPUID
XGETBV
платформа capability данные
↓
программа завершается до обычного запуск
рантайма
```

ИНЖЕНЕРНЫЙ ВЫВОД

Рантайм проверяет минимально требуемые инструкции до обычной инициализации.

Почему проверка CPU происходит до запуска рантайма

ЗАДАЧА И КОНТЕКСТ

Компилятор мог выпустить инструкции, которых нет на конкретном CPU. Запуск с неподходящими возможностями нельзя продолжать.

МЕХАНИЗМ

```
требуемый уровень GOAMD64
↓ ранняя проверка CPU
не поддерживается? → завершить
поддерживается? → osinit/schedinit
```

ИНЖЕНЕРНЫЙ ВЫВОД

Совместимость машинных инструкций проверяется до основного рантайма.

Это мост к 4.3

ЗАДАЧА И КОНТЕКСТ

В 4.3 компилятор выбирал ISA и оптимизации. Здесь стартовый код проверяет эти предположения на реальном процессоре.

МЕХАНИЗМ

```
GOARCH / GOAMD64
↓ компилятор сгенерировал код
↓ первый запуск на CPU
проверить обязательные возможности
```

ИНЖЕНЕРНЫЙ ВЫВОД

Выбранная при сборке архитектура должна соответствовать машине запуска.

Ранняя последовательность вызовов

ОПОРНАЯ МОДЕЛЬ

После подготовки TLS и проверки CPU `rt0_go` может перейти к функциям Go-рантайма.

АЛГОРИТМ

```
runtime.check
↓ runtime.args
↓ runtime.osinit
↓ runtime.schedinit
↓ newproc(runtime.main)
↓ runtime.mstart
```

ИНЖЕНЕРНЫЙ ВЫВОД

Механизмы поднимаются до первой обычной горутины.

runtime.args

ЗАДАЧА И КОНТЕКСТ

`runtime.args` получает `argc/argv` от загрузчика ОС и сохраняет их для дальнейшей инициализации.

МЕХАНИЗМ

```
ядро передало argc/argv
↓ _rt0_*
↓ runtime.args
позже → os.Args
```

ИНЖЕНЕРНЫЙ ВЫВОД

Данные аргументов появились еще до инициализации пакета `os`.

`args` — еще не пакет `os`

ЗАДАЧА И КОНТЕКСТ

Пакет `os` еще не инициализирован. Рантайм пока работает с исходными указателями из соглашений ОС.

МЕХАНИЗМ

```
ядро → argc/argv/envp
↓ runtime.args
↓ более поздняя настройка
os.Args / окружение
```

ИНЖЕНЕРНЫЙ ВЫВОД

Доступный Go API — позднее представление данных, полученных от ядра.

osinit

ЗАДАЧА И КОНТЕКСТ

`osinit` собирает базовую информацию платформы, которая потребуется планировщику и другим подсистемам.

МЕХАНИЗМ

```
параметры ОС
↓ число доступных CPU
↓ сведения о больших страницах
↓ платформенные механизмы
далее → schedinit
```

ИНЖЕНЕРНЫЙ ВЫВОД

Сначала получаем факты об ОС; потом выбираем размер набора P.

Что получает Linux при `osinit`?

ЗАДАЧА И КОНТЕКСТ

Linux `osinit` получает данные о процессорах, больших страницах и некоторых платформенных механизмах времени и случайности.

МЕХАНИЗМ

```
CPU на старте → numCPUStartup
большие страницы / vDSO
случайность / время
↓ schedinit
```

ИНЖЕНЕРНЫЙ ВЫВОД

Число процессоров будет использовано при выборе `GOMAXPROCS`.

Почему число процессоров нужен до планировщик

ЗАДАЧА И КОНТЕКСТ

Чтобы создать P, рантайм должен знать доступные процессорные ресурсы. `osinit` собирает исходные сведения.

МЕХАНИЗМ

```
ресурсы CPU в ОС
↓ runtime.osinit
↓ начальный GOMAXPROCS
↓ procresize
создать нужное число P
```

ИНЖЕНЕРНЫЙ ВЫВОД

Сведения об ОС и создание P — разные этапы запуска.

Go учитывает лимиты CPU контейнера при выборе GOMAXPROCS

ПРОБЛЕМА И КОНТЕКСТ

текущая версия Go выбирает по умолчанию GOMAXPROCS не только из простого NumCPU.

МЕХАНИЗМ

```
logical CPU count
привязка к CPU
Linux cgroup CPU throughput limit
```

ИНЖЕНЕРНЫЙ ВЫВОД

Это особенно важно в контейнерах.

Но решение о P еще не принято

ЗАДАЧА И КОНТЕКСТ

`osinit` лишь собирает данные. Конкретное число P позже определяется в `schedinit`.

МЕХАНИЗМ

```
osinit: CPU и платформа
↓ schedinit
GOMAXPROCS задан явно?
↓ вычислить число P
procresize
```

ИНЖЕНЕРНЫЙ ВЫВОД

Не приписываем `osinit` работу по созданию P.

Как `schedinit` поднимает рантайм

Аллокатор, метаданные, GC и P готовятся в строгом порядке.

```
stackinit → randinit → mallocinit
modulesinit → gcinit
GOMAXPROCS → procresize
```

Почему `schedinit` — не просто планировщик

ЗАДАЧА

Неверно: функция создает несколько очередей и завершается.

МЕХАНИЗМ

Она поднимает стековую подсистему, аллокатор, случайность, CPU, типы, интерфейсы, GC и набор P.

ИНЖЕНЕРНЫЙ ВЫВОД

Планировщик зависит от готовности других подсистем.

Порядок запуска указан прямо в исходниках

ЗАДАЧА И КОНТЕКСТ

В исходниках Go начальная цепочка описана явно, без необходимости восстанавливать ее по косвенным признакам.

МЕХАНИЗМ

```
osinit → schedinit
↓ создать и поставить в очередь G
↓ runtime.mstart
новая G исполняет runtime.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Это проверяемый путь из исходного кода рантайма.

Локи должны появиться раньше конкурентности

ЗАДАЧА И КОНТЕКСТ

Вскоре появится конкурентная работа нескольких M/P. Глобальные локи и их внутреннее состояние должны существовать заранее.

МЕХАНИЗМ

```
начало schedinit
↓ sched.lock / allglock
↓ allpLock / другие локи
↓ затем разрешается конкурентность
```

ИНЖЕНЕРНЫЙ ВЫВОД

Сначала готовим синхронизацию, потом запускаем конкурирующие потоки.

Почему рантайм инициализирует локи заранее

ЗАДАЧА И КОНТЕКСТ

Поднимать локи после появления нескольких работающих M было бы опасно: глобальные структуры уже начали бы использоваться конкурентно.

МЕХАНИЗМ

```
подготовить локи рантайма
↓ инициализировать их состояние
↓ создать P и планировщик
↓ разрешить работу нескольких M
```

ИНЖЕНЕРНЫЙ ВЫВОД

Механизмы синхронизации — предварительная зависимость планировщика.

Мир изначально считается остановленным

ЗАДАЧА И КОНТЕКСТ

В начале `schedinit` обычные G еще не выполняются. Рантайм отмечает состояние мира как остановленное.

МЕХАНИЗМ

```
мир еще не выполнял обычные G
↓ начальная STW-фаза
↓ последовательная настройка данных
позже → worldStarted()
```

ИНЖЕНЕРНЫЙ ВЫВОД

Первоначальная остановка мира упрощает построение глобальных структур.

Начальный STW отличается от обычного

ЗАДАЧА И КОНТЕКСТ

Обычный STW останавливает работающие горутины. При первоначальном запуске их еще нет.

МЕХАНИЗМ

```
обычный STW: работа → стоп → работа
стартовый STW: работы еще нет
↓ настройка всех подсистем
↓ появление P / возможность работы
```

ИНЖЕНЕРНЫЙ ВЫВОД

На старте STW — начальное состояние, а не остановка уже работающего приложения.

GODEBUG читается до запуска подсистем

ЗАДАЧА И КОНТЕКСТ

Некоторые параметры GODEBUG влияют на CPU-функции и политику планировщика. Их нужно знать до настройки этих механизмов.

МЕХАНИЗМ

```
прочитать ранние параметры GODEBUG
↓ настроить зависящие подсистемы
↓ продолжить schedinit
```

ИНЖЕНЕРНЫЙ ВЫВОД

Опцию, влияющую на создание подсистемы, нельзя применять только после ее создания.

Проверка метаданных компоновщика

ЗАДАЧА И КОНТЕКСТ

Компоновщик заранее подготовил `moduledata`. Перед использованием рантайм проверяет корректность этих данных.

МЕХАНИЗМ

```
компоновщик → moduledata
↓ moduledataverify
↓ modulesinit / типы / функции
```

ИНЖЕНЕРНЫЙ ВЫВОД

Метаданные файла проходят проверку до перехода в рабочие структуры.

Строгая цепочка внутри `schedinit`

ОПОРНАЯ МОДЕЛЬ

Порядок отражает зависимости: более поздняя подсистема использует подготовленные ранее механизмы.

АЛГОРИТМ

```
stackinit → randinit
↓ mallocinit → cpuinit
↓ maps.AlgInit
↓ mcommoninit(m0)
↓ modulesinit → typelinksinit
↓ itabsinit → stkobjinit
↓ gcinit
↓ GOMAXPROCS → procresize
```

ИНЖЕНЕРНЫЙ ВЫВОД

Поменять порядок местами без анализа зависимостей нельзя.

Почему порядок `schedinit` нельзя произвольно менять

ОПОРНАЯ МОДЕЛЬ

Поздние подсистемы опираются на подготовленные ранее данные и функции.

АЛГОРИТМ

```
случайность → хеши карт
аллокатор → внутренние объекты
метаданные модулей → типы/ITab
аллокатор + типы → GC
GC и планировщик → набор P
```

ИНЖЕНЕРНЫЙ ВЫВОД

Параллельный запуск зависимых подсистем усложнил бы первоначальную инициализацию.

stackinit

ЗАДАЧА И КОНТЕКСТ

Для обычных горутин потребуется выделение и учет стеков. `g0` пока использует стек ОС, но будущие G — нет.

МЕХАНИЗМ

```
stackinit
↓ подготовить стековые пулы
↓ следующая обычная G
может получить собственный стек
```

ИНЖЕНЕРНЫЙ ВЫВОД

Подсистема стеков готовится до первого вызова `newproc(runtime.main)`.

Стековый аллокатор уже знаком — теперь знаем момент запуска

ЗАДАЧА И КОНТЕКСТ

Рост и перемещение стека уже разобраны в лекции о планировщике. Здесь важно понять их место в последовательности запуска.

МЕХАНИЗМ

```
schedinit → stackinit
↓ стековый аллокатор готов
↓ newproc(runtime.main)
обычная G получает стек
```

ИНЖЕНЕРНЫЙ ВЫВОД

Переходим от механизма к моменту его появления.

`randinit`: случайность нужна самому рантайму

ЗАДАЧА И КОНТЕКСТ

`randinit` выполняется до аллокатора, алгоритмов карт и общей настройки M: они могут опираться на случайность.

МЕХАНИЗМ

```
randinit
↓ mallocinit
↓ maps.AlgInit
↓ mcommoninit(m0)
```

ИНЖЕНЕРНЫЙ ВЫВОД

Внутренний источник случайных значений — базовая подсистема рантайма.

Случайные значения — зависимость внутренних подсистем

ЗАДАЧА И КОНТЕКСТ

Рантайму нужны случайные значения для хеширования и собственного состояния M; это не пользовательский `math/rand`.

МЕХАНИЗМ

```
seed для хеширования
+ случайное состояние каждого M
+ внутренние защитные механизмы
↓ общий randinit
```

ИНЖЕНЕРНЫЙ ВЫВОД

Некоторые низкоуровневые подсистемы зависят от случайности уже при запуске.

mallocinit

ЗАДАЧА И КОНТЕКСТ

`mallocinit` включает аллокатор из лекции по памяти: арены, `mheap`, метаданные и распределение страниц.

МЕХАНИЗМ

```
schedinit
↓ mallocinit
↓ возможность аллокаций рантайма
позже → создание обычных G/P
```

ИНЖЕНЕРНЫЙ ВЫВОД

Аллокатор должен появиться раньше механизмов, которые выделяют память.

До `mallocinit` обычные аллокации недоступны

ЗАДАЧА И КОНТЕКСТ

До `mallocinit` рантайм не может свободно выделять память обычным способом.

МЕХАНИЗМ

```
поэтому используется:
статическая память m0/g0
+ первый стек ОС
+ специальные пути без аллокаций
```

ИНЖЕНЕРНЫЙ ВЫВОД

Необычный код раннего запуска связан с отсутствием готового аллокатора.

Аллокатор уже работает, но планировщик еще нет

ЗАДАЧА И КОНТЕКСТ

После `mallocinit` память выделять уже можно, но P и рабочий цикл планировщика еще не запущены.

МЕХАНИЗМ

```
аллокатор готов
↓ можно создать новые структуры
но: P еще не готовы
↓ procresize / mstart
```

ИНЖЕНЕРНЫЙ ВЫВОД

Готовность аллокатора и готовность планировщика — разные рубежи.

cpuinit

ЗАДАЧА И КОНТЕКСТ

`cpuinit` заполняет признаки возможностей процессора, необходимые для выбора доступных реализаций кода.

МЕХАНИЗМ

```
HasAVX / HasFMA
HasPOPCNT / HasSSE41
↓
проверки быстрых путей кода
```

ИНЖЕНЕРНЫЙ ВЫВОД

Это необязательные возможности CPU, а не только минимальная совместимость ISA.

Ранняя совместимость CPU и `cpuinit` решают разные задачи

ЗАДАЧА И КОНТЕКСТ

Ранняя проверка `GOAMD64` отвечает за обязательный набор инструкций. `cpuinit` заполняет сведения о дополнительных возможностях CPU.

МЕХАНИЗМ

```
ранняя проверка: обязательная ISA
↓
cpuinit: признаки CPU
↓ дополнительные быстрые пути
```

ИНЖЕНЕРНЫЙ ВЫВОД

Проверка минимальной совместимости и выбор оптимизаций — разные задачи.

Сгенерированный код может проверять возможности CPU

ЗАДАЧА И КОНТЕКСТ

После `cpuinit` некоторые пути сгенерированного кода могут выбирать реализацию по признакам CPU.

МЕХАНИЗМ

```
проверки признаков CPU
↓ разрешен нужный набор инструкций?
↓ выбрать соответствующую реализацию
```

ИНЖЕНЕРНЫЙ ВЫВОД

Флаги возможностей процессора должны быть корректными до обращения к ним.

maps.AlgInit

ЗАДАЧА И КОНТЕКСТ

Алгоритмы карт и хеширования используют внутреннее состояние рантайма и не готовы с самой первой инструкции.

МЕХАНИЗМ

```
randinit → cpuinit
↓ maps.AlgInit()
↓ разрешены штатные карты и хеши
```

ИНЖЕНЕРНЫЙ ВЫВОД

Даже фундаментальная конструкция языка имеет зависимости инициализации.

Даже `map` требует предварительной настройки рантайма

ЗАДАЧА И КОНТЕКСТ

Код рантайма уже выполняется, когда его алгоритмы карт еще не подготовлены. Это ограничение самого стартового этапа.

МЕХАНИЗМ

```
ранний код Go
↓ карты и хеши пока недоступны
maps.AlgInit()
↓ обычные операции с картами
```

ИНЖЕНЕРНЫЙ ВЫВОД

Возможность вызвать Go-код не означает готовность всех операций языка.

mcommoninit(m0)

ЗАДАЧА И КОНТЕКСТ

m0 пока был минимальным статическим объектом. mcommoninit(m0) выполняет общую настройку M.

МЕХАНИЗМ

```
m0 — начальное состояние
↓ mcommoninit(m0)
↓ регистрация M в структурах рантайма
готово представление потока
```

ИНЖЕНЕРНЫЙ ВЫВОД

Первый M проходит дополнительную инициализацию после создания аллокатора.

Что инициализируется у m0

ЗАДАЧА И КОНТЕКСТ

`mcommoninit` дополняет поля M: идентификатор, случайное состояние, профилирование и связи со списком M.

МЕХАНИЗМ

```
m0
↓ mcommoninit
идентификатор / случайные данные
профилирование / allm
↓ полноценный объект M
```

ИНЖЕНЕРНЫЙ ВЫВОД

Идентичность `m0` создана раньше, но его обычные поля настраиваются здесь.

Почему жизненный цикл m0 особенный

ЗАДАЧА И КОНТЕКСТ

Обычный M создается решением рантайма и новым потоком ОС. m0 отличается: поток был создан ядром еще до начала Go.

МЕХАНИЗМ

```
обычный M: newm → поток ОС
m0: поток ОС уже существует
↓ статический объект m0
↓ mcommoninit
```

ИНЖЕНЕРНЫЙ ВЫВОД

Первый M нельзя описывать стандартным путем создания дополнительных M.

`modulesinit`: метаданные компоновщика оживают

ЗАДАЧА И КОНТЕКСТ

Компоновщик создал таблицы функций и типов. Теперь `modulesinit` делает их доступными активному рантайму.

МЕХАНИЗМ

```
moduledata из исполняемого файла
↓ modulesinit
активные метаданные модулей
```

ИНЖЕНЕРНЫЙ ВЫВОД

Отображенные байты и рабочие структуры рантайма — разные состояния.

От `moduledata` в файле к данным рантайма

ЗАДАЧА И КОНТЕКСТ

Рантайм использует статические данные, подготовленные компоновщиком, но сначала должен связать их в рабочие списки.

МЕХАНИЗМ

```
компоновщик → moduledata
↓ загрузчик отобразил байты
runtime.modulesinit
↓ activeModules
```

ИНЖЕНЕРНЫЙ ВЫВОД

Это переход от результатов компоновки к структурам выполняющейся программы.

`typelinksinit`: ссылки на типы

ЗАДАЧА И КОНТЕКСТ

`typelinksinit` подготавливает связи между описателями типов, нужные рефлексии и поиску типов между модулями.

МЕХАНИЗМ

```
описатели типов из файла
↓ typelinksinit
связи типов / рефлексия
поиск типов между модулями
```

ИНЖЕНЕРНЫЙ ВЫВОД

Метаданные типов требуют инициализации до обычного использования.

`itabsinit`: подготовка таблиц интерфейсов

ЗАДАЧА И КОНТЕКСТ

`itabsinit` подготавливает общие структуры таблиц методов интерфейсов после инициализации типов.

МЕХАНИЗМ

```
метаданные компилятора/компоновщика
↓ typelinksinit
↓ itabsinit
обычный вызов через интерфейс
```

ИНЖЕНЕРНЫЙ ВЫВОД

Рабочие структуры интерфейсов зависят от корректной информации о типах.

Интерфейсам нужны и статические, и рабочие структуры

ЗАДАЧА И КОНТЕКСТ

Для интерфейсов часть описаний существует в файле заранее, а часть общих таблиц готовится при запуске.

МЕХАНИЗМ

```
статические описатели типов / методов
+ структуры рантайма
↓ itabsinit
готова инфраструктура ITab
```

ИНЖЕНЕРНЫЙ ВЫВОД

Нельзя считать все таблицы интерфейсов полностью готовыми при входе в `rt0`.

`stkobjinit`: данные для сканирования стеков

ЗАДАЧА И КОНТЕКСТ

`stkobjinit` подготавливает сведения об объектах на стеках перед началом сборки мусора.

МЕХАНИЗМ

```
метаданные объектов стека
↓ stkobjinit
↓ корректное сканирование стеков GC
```

ИНЖЕНЕРНЫЙ ВЫВОД

GC должен точно понимать указатели стека с первого полноценного цикла.

От `argc/argv` до `os.Args`

ОПОРНАЯ МОДЕЛЬ

`runtime.args` запоминает переданные ядром указатели. Позже появляются структуры, доступные Go-коду.

АЛГОРИТМ

```
ядро: argv/envp
↓ _rt0_*
↓ runtime.args
↓ goargs / goenvs
os.Args и окружение
```

ИНЖЕНЕРНЫЙ ВЫВОД

Данные запуска переходят из соглашений ОС в структуры рантайма.

Рантайм проверяет часть окружения до `main`

ЗАДАЧА И КОНТЕКСТ

Во время `schedinit` рантайм выполняет ранние проверки окружения и дескрипторов процесса.

МЕХАНИЗМ

```
secure()
checkfds()
↓ нормализация части состояния ОС
↓ дальнейшая инициализация
```

ИНЖЕНЕРНЫЙ ВЫВОД

Часть проверок безопасности выполняется до пользовательского `main`.

gcinit

ЗАДАЧА И КОНТЕКСТ

`gcinit` создает рабочее состояние сборщика мусора после настройки аллокатора и метаданных.

МЕХАНИЗМ

```
schedinit
↓ gcinit
готова инфраструктура GC
позднее → gcenable
```

ИНЖЕНЕРНЫЙ ВЫВОД

Готовность сборщика и запуск его фоновых горутин разделены.

`gcinit` и `gcenable` — две разные границы

ЗАДАЧА

`gcinit` подготавливает состояние сборщика во время `schedinit`.

МЕХАНИЗМ

`gcenable` запускается позже внутри `runtime.main`, когда готов планировщик.

ИНЖЕНЕРНЫЙ ВЫВОД

Инициализация подсистемы не равна разрешению ее фоновой работы.

Почему не включить GC сразу

ЗАДАЧА И КОНТЕКСТ

Фоновую работу GC нельзя запускать раньше, чем готовы аллокатор, типы, P и планировщик.

МЕХАНИЗМ

```
сначала: аллокатор и метаданные
↓ gcinit
↓ создать P и запустить планировщик
потом: gcenable
```

ИНЖЕНЕРНЫЙ ВЫВОД

Разделение подготовки и фоновой работы упрощает первоначальный запуск.

Резервный стек для ранней аварии

ЗАДАЧА И КОНТЕКСТ

Если ранняя авария повредила обычный системный стек, диагностика на нем самом может оказаться невозможной.

МЕХАНИЗМ

```
авария стека g0 / morestack
↓ обычный стек ненадежен
резервный стек аварийной обработки
```

ИНЖЕНЕРНЫЙ ВЫВОД

Рантайм предусматривает отдельный стек для некоторых критических ошибок.

Что должно быть готово до первого P?

ЗАДАЧА И КОНТЕКСТ

Перед созданием P должны работать стековая подсистема, аллокатор, типы, алгоритмы карт и начальное состояние GC.

МЕХАНИЗМ

```
стеки / память / случайность
CPU / карты / M
метаданные типов / GC
↓
переход к GOMAXPROCS и P
```

ИНЖЕНЕРНЫЙ ВЫВОД

Число P выбирается после подготовки нижележащих ресурсов.

Как задается начальное GOMAXPROCS

ЗАДАЧА И КОНТЕКСТ

В `schedinit` сначала проверяется явная настройка окружения `GOMAXPROCS`.

МЕХАНИЗМ

```
GOMAXPROCS=N и N > 0?
↓ да → использовать N
↓ пометить значение как заданное явно
↓ вызвать procresize
```

ИНЖЕНЕРНЫЙ ВЫВОД

Явная настройка отличается от автоматически выбранного значения по умолчанию.

Что если GOMAXPROCS не задан?

ЗАДАЧА И КОНТЕКСТ

Без явного значения рантайм вызывает `defaultGOMAXPROCS(numCPUStartup)` и учитывает текущую платформенную политику.

МЕХАНИЗМ

```
число CPU / привязка к CPU
+ лимит CPU контейнера
↓ defaultGOMAXPROCS
↓ нужное число P
```

ИНЖЕНЕРНЫЙ ВЫВОД

Точная политика выбора по умолчанию зависит от версии Go.

Почему число P нужно выбрать уже при запуске

ЗАДАЧА И КОНТЕКСТ

Число P определяет размер набора структур планировщика: локальные очереди, кеши, таймеры и другие ресурсы P.

МЕХАНИЗМ

```
начальное GOMAXPROCS
↓ procresize
↓ allp / очереди / кеши
↓ готовность P к работе
```

ИНЖЕНЕРНЫЙ ВЫВОД

Значение `GOMAXPROCS` влияет на физическую структуру рантайма.

procresize

ЗАДАЧА И КОНТЕКСТ

`procresize` приводит число P к нужному значению. Во время этой операции удерживается лок планировщика, а мир остановлен.

МЕХАНИЗМ

```
sched.lock удерживается
+ мир остановлен
↓ procresize(nprocs)
↓ изменить набор P
```

ИНЖЕНЕРНЫЙ ВЫВОД

Изменять количество P безопаснее при остановленной работе Go-кода.

`procresize` создает набор P

ОПОРНАЯ МОДЕЛЬ

Планировщик уже знает целевое число P из политики `GOMAXPROCS`.

АЛГОРИТМ

```
нужное число P
↓ расширить allp
↓ создать недостающие P
↓ pp.init(i)
↓ связать P с M
↓ обновить gomaxprocs
```

ИНЖЕНЕРНЫЙ ВЫВОД

После этой стадии P способны выполнять обычный Go-код.

Здесь появляются обычные P

ЗАДАЧА И КОНТЕКСТ

После `procresize` доступны P0…P(N−1), способные участвовать в исполнении обычного Go-кода.

МЕХАНИЗМ

```
до procresize: P нет
↓ создать и настроить P
после: P0, P1, …, P(N−1)
```

ИНЖЕНЕРНЫЙ ВЫВОД

P возникают внутри `schedinit`, а не в момент входа процесса.

`P.init` подготавливает структуры каждого P

ЗАДАЧА И КОНТЕКСТ

Инициализация каждого P подготавливает знакомые структуры: локальную очередь, `runnext`, `mcache`, таймеры и состояние GC.

МЕХАНИЗМ

```
pp.init(i)
↓ локальная очередь и runnext
↓ mcache / таймеры
↓ локальные структуры GC
```

ИНЖЕНЕРНЫЙ ВЫВОД

Так в жизни процесса появляется ранее изученная структура P.

Когда рантайм впервые способен исполнять Go-код

ЗАДАЧА И КОНТЕКСТ

В исходниках рантайм отмечает: после подготовки P мир уже способен исполнять Go-код.

МЕХАНИЗМ

```
schedinit
↓ procresize
↓ набор P готов
worldStarted()
```

ИНЖЕНЕРНЫЙ ВЫВОД

Это рубеж возможности исполнения, но планировщик еще не выбрал первую G.

«Мир начал работать» — еще не `main.main`

ЗАДАЧА И КОНТЕКСТ

Существование P еще не означает, что началось выполнение пользовательского кода или даже `runtime.main`.

МЕХАНИЗМ

```
P готовы
≠ schedule() уже выбрал G
≠ main.main уже вызван
↓ дальше newproc → mstart
```

ИНЖЕНЕРНЫЙ ВЫВОД

Не смешиваем готовность P, работу планировщика и начало приложения.

Три рубежа готовности рантайма

ОПОРНАЯ МОДЕЛЬ

Слово «запущен» скрывает разные технические состояния.

АЛГОРИТМ

```
A: m0/g0 существуют
↓
B: procresize создал P
↓
C: mstart → schedule исполняет обычную G
```

ИНЖЕНЕРНЫЙ ВЫВОД

Выполнение пользовательского `main` начинается существенно позже.

Результат `schedinit`: основа готова, G еще нет

ЗАДАЧА И КОНТЕКСТ

`schedinit` завершил создание аллокатора, метаданных, состояния GC и набора P.

МЕХАНИЗМ

```
m0/g0 + аллокатор
+ типы / интерфейсы / GC
+ P и очереди
↓ обычная G(runtime.main)
еще только должна появиться
```

ИНЖЕНЕРНЫЙ ВЫВОД

Основа готова, но первый запуск обычной G еще впереди.

Первая обычная горутина

После подготовки P нужно создать `G(runtime.main)` и запустить цикл планировщика.

```
newproc(runtime.main)
mstart → schedule
g0 → G(runtime.main)
```

Возвращаемся в `rt0_go`

ЗАДАЧА И КОНТЕКСТ

После `schedinit` стартовый код снова выполняется в `rt0_go` и переходит к созданию первой обычной G.

МЕХАНИЗМ

```
runtime.args
↓ runtime.osinit
↓ runtime.schedinit
newproc(runtime.main)
```

ИНЖЕНЕРНЫЙ ВЫВОД

Первая обычная горутина создается после подготовки аллокатора и P.

Первая обычная G начнет с `runtime.main`

ПРОБЛЕМА И КОНТЕКСТ

и передает его `newproc`.

МЕХАНИЗМ

```
main.main
↓
runtime.main
↓
runtime.mainPC
```

ИНЖЕНЕРНЫЙ ВЫВОД

Первая обычная горутина рантайма еще не пользовательская.

Почему пользовательский `main` пока не запускается

ЗАДАЧА И КОНТЕКСТ

До пользовательского `main` рантайм обязан запустить системный монитор, включить фоновый GC и выполнить инициализацию пакетов.

МЕХАНИЗМ

```
первая G → runtime.main
↓ sysmon / gcenable
↓ задачи init
потом → main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Пользовательский `main` должен получить уже готовую среду выполнения.

Почему теперь `newproc` может работать нормально

ЗАДАЧА И КОНТЕКСТ

После `schedinit` уже готовы стековый аллокатор, память, P и локальные очереди.

МЕХАНИЗМ

```
schedinit завершен
↓ обычный newproc
↓ выделить стек G
↓ поставить G в очередь
```

ИНЖЕНЕРНЫЙ ВЫВОД

Создание `G(runtime.main)` использует известный механизм рантайма.

Но `newproc` второй раз не разбираем

ПРОБЛЕМА И КОНТЕКСТ

Механика `newproc1`, `gostartcallfn`, runq уже изучена.

МЕХАНИЗМ

```
newproc(runtime.main)
↓
G создана
↓
_Grunnable
↓
поставлена в очередь
```

ИНЖЕНЕРНЫЙ ВЫВОД

Механизм создания G здесь тот же, что изучен в `углубленной лекции про рантайм`.

Почему готовая G пока еще не выполняется

ЗАДАЧА И КОНТЕКСТ

`newproc` создал G и поместил ее в очередь, но первый поток ОС еще выполняет стартовый код на `g0`.

МЕХАНИЗМ

```
G(runtime.main) готова к выполнению
пока rt0_go исполняется на g0
↓ необходим переход в mstart
```

ИНЖЕНЕРНЫЙ ВЫВОД

Готовность G и фактическое исполнение — разные состояния.

После первой G стартует M

ОПОРНАЯ МОДЕЛЬ

`newproc` поставил G с `runtime.main` в очередь, но стартовый поток еще исполняет `g0`.

АЛГОРИТМ

```
newproc(runtime.main)
↓ G готова к выполнению
runtime.mstart
↓ mstart0 → mstart1
↓ schedule()
```

ИНЖЕНЕРНЫЙ ВЫВОД

Первую обычную G запускает уже работающий планировщик.

`mstart` запускает жизненный цикл M

ЗАДАЧА И КОНТЕКСТ

`mstart` — низкоуровневая точка входа в обычный жизненный цикл M.

МЕХАНИЗМ

```
runtime.mstart
↓ mstart0
↓ mstart1
↓ schedule()
```

ИНЖЕНЕРНЫЙ ВЫВОД

Теперь начальный поток входит в рабочий цикл планировщика.

`mstart0` проверяет системный стек

ЗАДАЧА И КОНТЕКСТ

`mstart0` проверяет границы системного стека. Для `m0` это все еще исходный стек ОС.

МЕХАНИЗМ

```
m0 → исходный системный стек
↓ mstart0
↓ окончательная проверка защиты стека
↓ mstart1
```

ИНЖЕНЕРНЫЙ ВЫВОД

При переходе к обычной работе M должны выполняться инварианты стека.

Зачем снова проверять границы системного стека

ЗАДАЧА И КОНТЕКСТ

Ранний `rt0_go` задавал первоначальные границы. `mstart0` завершает настройку стека для обычной работы рантайма.

МЕХАНИЗМ

```
предварительные границы g0
↓ mstart0
окончательная проверка / защита
↓ mstart1
```

ИНЖЕНЕРНЫЙ ВЫВОД

Начальная оценка границ стека не заменяет полноценную настройку M.

mstart1

ЗАДАЧА И КОНТЕКСТ

`mstart1` выполняется на `g0` и подготавливает M к штатному циклу планировщика.

МЕХАНИЗМ

```
mstart1
↓ asminit / minit
↓ при необходимости настройки m0
↓ schedule()
```

ИНЖЕНЕРНЫЙ ВЫВОД

После настройки M можно впервые выбрать обычную G.

g0.sched: состояние системных переключений

ЗАДАЧА И КОНТЕКСТ

Рантайм сохраняет сведения о системном контексте g0, необходимые для переключений и трассировок.

МЕХАНИЗМ

```
g0.sched
↓ контекст M на системном стеке
↓ корректные переходы в планировщик
```

ИНЖЕНЕРНЫЙ ВЫВОД

Рабочий цикл планировщика требует корректного системного контекста M.

asminit и minit

ЗАДАЧА И КОНТЕКСТ

asminit и minit завершают платформенную настройку потока: сигналы, TLS и служебные данные.

МЕХАНИЗМ

```
mstart1
↓ asminit()
↓ minit()
↓ schedule()
```

ИНЖЕНЕРНЫЙ ВЫВОД

После платформенной настройки поток ОС может исполнять штатную работу Go.

Почему m0 требует отдельной настройки

ЗАДАЧА И КОНТЕКСТ

m0 не проходил обычный путь newm: его поток уже существовал. Поэтому часть настроек выполняется отдельно.

МЕХАНИЗМ

```
первый M: m0
↓ специальная настройка сигналов
↓ завершить контекст потока
↓ schedule()
```

ИНЖЕНЕРНЫЙ ВЫВОД

Отдельный путь m0 не следует путать с созданием обычного M.

И наконец `schedule()`

ЗАДАЧА И КОНТЕКСТ

После настройки `mstart1` впервые входит в штатную функцию `schedule()`.

МЕХАНИЗМ

```
mstart1
↓ schedule()
↓ выбрать готовую G
↓ execute
```

ИНЖЕНЕРНЫЙ ВЫВОД

С этого места применяется планировщик из `углубленной лекции про рантайм`.

`schedule()` уже изучен: не повторяем планировщик

ЗАДАЧА И КОНТЕКСТ

Алгоритм `schedule → findRunnable → execute` уже изучен. Здесь важно только место первого входа в него.

МЕХАНИЗМ

```
mstart1
↓ schedule
↓ findRunnable
↓ execute(G(runtime.main))
```

ИНЖЕНЕРНЫЙ ВЫВОД

Подробности очередей и вытеснения повторно разбирать не нужно.

Какая готовый к выполнению G сейчас самая важная

ЗАДАЧА И КОНТЕКСТ

`newproc` только что поставил `G(runtime.main)` в очередь готовых к выполнению.

МЕХАНИЗМ

```
готовая G(runtime.main)
↓ первый schedule()
↓ выбранная G
↓ исполнение runtime.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Планировщик получает свою первую обычную работу.

Первый переход с g0 на обычную G

ОПОРНАЯ МОДЕЛЬ

До этого весь запуск выполнялся на служебной G первого M.

АЛГОРИТМ

```
первый поток ОС → m0/g0
↓ завершить schedinit
newproc(runtime.main)
↓ mstart → schedule
m0 + P → G(runtime.main)
```

ИНЖЕНЕРНЫЙ ВЫВОД

Именно здесь начинается привычная модель G/M/P.

Граница, после которой работает модель G/M/P

ЗАДАЧА И КОНТЕКСТ

Теперь одновременно существуют M, P, готовая обычная G и полноценный механизм исполнения.

МЕХАНИЗМ

```
m0 + P
↓ schedule()
↓ G(runtime.main)
обычная модель G/M/P работает
```

ИНЖЕНЕРНЫЙ ВЫВОД

От этой точки можно пользоваться выводами лекции `углубленной лекции про рантайм` без оговорок о запуске.

От `runtime.main` к пользовательскому коду

Системный монитор, задания инициализации, включение GC и вызов `main.main`.

```
sysmon / LockOSThread
runtime_inittasks → gcenable
m.inittasks → main.main
```

Но `runtime.main` все еще не пользовательский `main`

ЗАДАЧА И КОНТЕКСТ

Первая обычная G исполняет `runtime.main`. Пользовательский `main.main` еще не вызван.

МЕХАНИЗМ

```
runtime.main
↓ системные фоновые службы
↓ задачи инициализации рантайма
↓ инициализация пакетов
потом → main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

`runtime.main` готовит все, что должно существовать до пользовательского кода.

Почему `runtime.main` запускается именно на `m0`

ЗАДАЧА И КОНТЕКСТ

`runtime.main` проверяет, что выполняется на начальном M (`m0`), связанном со стартовым потоком ОС.

МЕХАНИЗМ

```
getg().m == &m0?
да → продолжить запуск
нет → внутренняя авария
```

ИНЖЕНЕРНЫЙ ВЫВОД

Стадия инициализации должна начаться на стартовом потоке ОС.

mainStarted

ЗАДАЧА И КОНТЕКСТ

В начале `runtime.main` устанавливается признак `mainStarted`, отмечающий запуск обычного мира G/M/P.

МЕХАНИЗМ

```
G(runtime.main)
↓ mainStarted = true
↓ обычное пробуждение/создание M
```

ИНЖЕНЕРНЫЙ ВЫВОД

После этой границы рантайм может шире использовать штатную многопоточность.

Почему рано создавать другие M

ЗАДАЧА И КОНТЕКСТ

До `mainStarted` рантайм специально ограничивает обычную конкурентную работу других M, пока базовые подсистемы еще не полностью подготовлены.

МЕХАНИЗМ

```
подготовка основных структур
↓ mainStarted
↓ можно запускать обычные M
↓ выполнять Go-код конкурентно
```

ИНЖЕНЕРНЫЙ ВЫВОД

Преждевременная конкуренция усложнила бы первоначальную инициализацию.

`sysmon` появляется уже в `runtime.main`

ОПОРНАЯ МОДЕЛЬ

Нельзя запускать полноценный монитор, пока еще не создан рабочий мир планировщика.

АЛГОРИТМ

```
schedinit → procresize
↓ mstart → schedule
↓ G(runtime.main)
↓ newm(sysmon, nil, -1)
```

ИНЖЕНЕРНЫЙ ВЫВОД

Подсистемы фона включаются после создания основы.

Почему `sysmon` не запускают в `schedinit`

ЗАДАЧА И КОНТЕКСТ

В `schedinit` мир еще считается остановленным, а P только создаются. Системный монитор лучше запустить позднее.

МЕХАНИЗМ

```
schedinit: подготовка структур
↓ mstart → schedule
↓ G(runtime.main)
↓ запустить sysmon
```

ИНЖЕНЕРНЫЙ ВЫВОД

Системный монитор появляется после рабочего планировщика, а не до него.

Не перечитываем sysmon

ЗАДАЧА И КОНТЕКСТ

Алгоритм sysmon уже знаком: он помогает с таймерами, вытеснением и возвратом P.

МЕХАНИЗМ

```
точка создания: runtime.main
↓ newm(sysmon, nil, -1)
↓ фоновая системная работа
```

ИНЖЕНЕРНЫЙ ВЫВОД

Здесь изучаем момент запуска, а не повторяем внутренности монитора.

`runtime.main` временно закрепляет поток ОС

ЗАДАЧА И КОНТЕКСТ

`runtime.main` временно вызывает `lockOSThread()` перед инициализацией пакетов.

МЕХАНИЗМ

```
главная G(runtime.main)
↓ lockOSThread()
↓ выполнить задания инициализации
↓ освободить поток после init
```

ИНЖЕНЕРНЫЙ ВЫВОД

Инициализация проходит на стартовом потоке ОС.

Зачем инициализация пакетов идет на стартовом потоке

ЗАДАЧА И КОНТЕКСТ

Некоторые платформенные API требуют вызовов на главном потоке ОС. Поэтому Go выполняет `init` на стартовом потоке.

МЕХАНИЗМ

```
runtime.main закрепляет поток ОС
↓ инициализация пакетов
↓ код init() на том же потоке
↓ дальнейший main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Привязка потока для `init` — часть публично описанного поведения Go.

Гарантия `runtime.LockOSThread` для `init`

ЗАДАЧА И КОНТЕКСТ

`runtime.LockOSThread` документирует выполнение `init` на стартовом потоке ОС. `init()` может дополнительно закрепить этот поток.

МЕХАНИЗМ

```
init() вызывает LockOSThread
↓ привязка главной G сохраняется
↓ возможен запуск main.main
на том же потоке
```

ИНЖЕНЕРНЫЙ ВЫВОД

Публичная гарантия API не зависит от внутреннего имени `runtime.main`.

`runtime.main` закрепляет стартовый поток на время `init`

ОПОРНАЯ МОДЕЛЬ

Для некоторых платформ нужно выполнять инициализацию на стартовом потоке ОС.

АЛГОРИТМ

```
runtime.main
↓ lockOSThread()
↓ runtime_inittasks / m.inittasks
↓ после init: unlockOSThread()
↓ main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Если пользовательский `init` сам закрепил поток, привязка может сохраниться.

Горутины, созданные из `init`, не закрепляются автоматически

ЗАДАЧА И КОНТЕКСТ

Привязанной остается главная G, исполняющая инициализацию. Горутины, созданные через `go`, получают обычное планирование.

МЕХАНИЗМ

```
главная G: lockOSThread()
↓ init()
↓ go worker()
новая G планируется независимо
```

ИНЖЕНЕРНЫЙ ВЫВОД

Закрепление одной G не распространяется автоматически на созданные ею горутины.

Откуда берется время начала инициализации

ЗАДАЧА И КОНТЕКСТ

Рантайм фиксирует момент входа в этап инициализации, чтобы измерять продолжительность подготовки пакетов.

МЕХАНИЗМ

```
runtime.main
↓ runtimeInitTime
↓ включить inittrace
↓ задачи инициализации
```

ИНЖЕНЕРНЫЙ ВЫВОД

У измерения `init` есть конкретная временная граница.

GODEBUG=inittrace=1: измеряем init

ЗАДАЧА И КОНТЕКСТ

Включенный `GODEBUG=inittrace=1` позволяет увидеть стоимость инициализации отдельных пакетов.

МЕХАНИЗМ

```
GODEBUG=inittrace=1
↓ пакет / время
↓ байты / аллокации
↓ найти дорогой init
```

ИНЖЕНЕРНЫЙ ВЫВОД

Это измеряет инициализацию пакетов, а не работу загрузчика ELF.

Почему `inittrace` включается именно в `runtime.main`

ЗАДАЧА И КОНТЕКСТ

Раньше еще не готов обычный аллокатор и первая G; позже начало `init` уже было бы пропущено.

МЕХАНИЗМ

```
до runtime.main — слишком рано
runtime.main — начать измерение
после init — выключить
```

ИНЖЕНЕРНЫЙ ВЫВОД

Трассировка инициализации включается на подходящем этапе запуска.

Сначала выполняется инициализация самого рантайма

ЗАДАЧА И КОНТЕКСТ

`runtime.main` сначала вызывает `doInit(runtime_inittasks)`, а только потом переходит к пользовательским пакетам.

МЕХАНИЗМ

```
G(runtime.main)
↓ doInit(runtime_inittasks)
↓ готовность внутренних пакетов
позже → другие inittasks
```

ИНЖЕНЕРНЫЙ ВЫВОД

Внутренние задачи инициализации рантайма предшествуют остальным пакетам.

Список `runtime_inittasks` сформирован компоновщиком

ЗАДАЧА И КОНТЕКСТ

`runtime_inittasks` строится компоновщиком. Рантайму не нужно заново вычислять порядок зависимостей.

МЕХАНИЗМ

```
компилятор: сведения об init
↓ компоновщик: runtime_inittasks
↓ runtime.main / doInit
```

ИНЖЕНЕРНЫЙ ВЫВОД

Граф запуска пакетов частично вычислен еще до исполнения программы.

Компилятор, компоновщик и рантайм делят обязанности

ЗАДАЧА И КОНТЕКСТ

Инструментарий распределяет обязанности: компилятор знает семантику, компоновщик собирает порядок, рантайм выполняет задачи.

МЕХАНИЗМ

```
компилятор → функции init
↓ компоновщик → последовательность
↓ рантайм → doInit
```

ИНЖЕНЕРНЫЙ ВЫВОД

Вычисление неизменного порядка заранее упрощает работу программы.

initTask: единица инициализации пакета

ЗАДАЧА И КОНТЕКСТ

initTask — единица инициализации пакета: переменные пакета и его функции init().

МЕХАНИЗМ

```
описатели задач от компоновщика
↓ doInit(task)
↓ инициализация переменных
↓ функции init()
```

ИНЖЕНЕРНЫЙ ВЫВОД

Каждый пакет должен пройти свою инициализацию до зависящих пакетов.

Инициализация пакета включает переменные и функции

GO / КОМАНДА

```
var cfg = loadConfig()

func init() {
    register(cfg)
}
```

ЧТО ПРОИСХОДИТ

Выполняется `loadConfig()`, затем `register(cfg)`. Тяжелая работа может быть скрыта в глобальной переменной.

ИНЖЕНЕРНЫЙ ВЫВОД

Пустой `init()` еще не означает дешевую инициализацию пакета.

Тяжелый код может скрываться в глобальных переменных

ЗАДАЧА И КОНТЕКСТ

Тяжелая работа может быть написана в объявлении переменной и выполниться до `main`, даже без явного `init()`.

МЕХАНИЗМ

```
var client = newHeavyClient()
↓ инициализация пакета
↓ скрытая цена запуска
потом → main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

При поиске медленного запуска проверяем и глобальные инициализаторы.

Два момента жизни GC

ОПОРНАЯ МОДЕЛЬ

В `schedinit` готовится состояние GC, но фоновые механизмы запускаются только позднее.

АЛГОРИТМ

```
schedinit → gcinit
↓ создать P / запустить планировщик
runtime.main
↓ задачи инициализации рантайма
gcenable
↓ фоновая работа GC
```

ИНЖЕНЕРНЫЙ ВЫВОД

Запуск GC разделен на подготовку и включение.

`gcenable`: теперь разрешена фоновая работа GC

ЗАДАЧА И КОНТЕКСТ

`gcenable` включается после готовности планировщика и внутренних задач инициализации.

МЕХАНИЗМ

```
gcinit: состояние сборщика
↓ обычная G и P
runtime.main → gcenable
↓ фоновая работа GC
```

ИНЖЕНЕРНЫЙ ВЫВОД

Подготовка GC и начало фоновой работы разнесены по времени.

Почему фоновый GC включается только после планировщика

ЗАДАЧА И КОНТЕКСТ

Фоновые горутины GC требуют аллокатор, метаданные и работающий планировщик.

МЕХАНИЗМ

```
аллокатор + типы + GC state
↓ набор P / schedule()
↓ gcenable()
↓ фоновая работа GC
```

ИНЖЕНЕРНЫЙ ВЫВОД

Порядок включения GC определяется его зависимостями от других подсистем.

Подсистема сначала создается, затем начинает работать

ЗАДАЧА И КОНТЕКСТ

Для ряда подсистем рантайм сначала создает структуры, а лишь позднее разрешает конкурентную работу.

МЕХАНИЗМ

```
структуры инициализированы
↓ глобальное состояние корректно
↓ планировщик запущен
↓ фоновые механизмы включены
```

ИНЖЕНЕРНЫЙ ВЫВОД

Этот прием помогает избежать циклических зависимостей запуска.

Обновление GOMAXPROCS начинается позже запуска

ЗАДАЧА И КОНТЕКСТ

После `gcenable` рантайм может включить периодический пересчет значения `GOMAXPROCS` по умолчанию.

МЕХАНИЗМ

```
при запуске: выбрать число P
↓ после инициализации рантайма
разрешить периодическую проверку
↓ при изменении лимитов скорректировать P
```

ИНЖЕНЕРНЫЙ ВЫВОД

Пересчет требует уже работающих механизмов планировщика и фоновой работы.

Начальное число P не обязательно останется неизменным

ЗАДАЧА И КОНТЕКСТ

Начальное `GOMAXPROCS` задано в `schedinit`, но доступный CPU-лимит контейнера может меняться.

МЕХАНИЗМ

```
schedinit: первоначальное значение
↓ меняется привязка CPU / квота cgroup
↓ рантайм проверяет новую политику
возможен procresize позднее
```

ИНЖЕНЕРНЫЙ ВЫВОД

В текущем Go значение по умолчанию не обязательно фиксировано на весь процесс.

cgo: согласование рантайма с потоками C

ЗАДАЧА И КОНТЕКСТ

С cgo необходимо согласовать потоки C, TLS, callback-вызовы и завершение инициализации рантайма.

МЕХАНИЗМ

```
runtime.main
↓ проверки cgo / pthread
↓ подготовить callback-взаимодействие
↓ дождаться завершения init
```

ИНЖЕНЕРНЫЙ ВЫВОД

cgo добавляет дополнительное согласование между двумя моделями потоков.

Почему cgo-вызовы ждут завершения инициализации

ЗАДАЧА И КОНТЕКСТ

Поток C не должен попасть в обычный Go callback, пока рантайм еще проходит критическую инициализацию.

МЕХАНИЗМ

```
поток C вызывает Go слишком рано
↓ ждать mainInitDoneChan
↓ mainInitDone = true
↓ разрешить нормальный callback
```

ИНЖЕНЕРНЫЙ ВЫВОД

Завершение инициализации имеет явный сигнал для cgo.

Компоновщик уже вычислил порядок `init`

ОПОРНАЯ МОДЕЛЬ

В `runtime.main` готовый список задач инициализации выполняется согласно зависимостям пакетов.

АЛГОРИТМ

```
компилятор: функции инициализации
↓ компоновщик: inittasks
↓ runtime.main
↓ doInit(m.inittasks)
↓ main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Рантайму не нужно заново строить граф импортов.

Почему рантайм предусматривает несколько модулей

ЗАДАЧА И КОНТЕКСТ

В обычном исполняемом файле метаданные модулей просты. Плагины и другие режимы могут добавить несколько модулей.

МЕХАНИЗМ

```
модули содержат свои inittasks
↓ runtime.main проходит список модулей
↓ выполняет инициализацию
```

ИНЖЕНЕРНЫЙ ВЫВОД

Рантайм использует общую модель, рассчитанную не только на один модуль.

Зависимости и переменные инициализируются по порядку

ЗАДАЧА И КОНТЕКСТ

Перед текущим пакетом должны быть готовы зависимости, затем его переменные и функции `init`.

МЕХАНИЗМ

```
инициализация зависимостей
↓ переменные текущего пакета
↓ init() текущего пакета
↓ зависящий пакет
```

ИНЖЕНЕРНЫЙ ВЫВОД

Порядок инициализации пакетов задан семантикой Go.

Инициализацию пакетов выполняет одна главная G

ЗАДАЧА И КОНТЕКСТ

Сам список задач `init` выполняется последовательно главной горутиной, а не по горутине на пакет.

МЕХАНИЗМ

```
главная G
↓ doInit(package A)
↓ doInit(package B)
↓ doInit(package C)
```

ИНЖЕНЕРНЫЙ ВЫВОД

Последовательность задач упрощает порядок зависимостей между пакетами.

`init()` может создавать конкурентную работу

GO / КОМАНДА

```
func init() {
    go background()
}
```

ЧТО ПРОИСХОДИТ

Сама последовательность инициализации идет в главной G, но созданная горутина может выполняться параллельно.

ИНЖЕНЕРНЫЙ ВЫВОД

«Инициализация последовательная» не означает «конкурентности нет».

Последовательность `init` не означает отсутствия конкурентности

ЗАДАЧА И КОНТЕКСТ

Основная цепочка `init` последовательна, однако сами инициализаторы могут создавать другие горутины.

МЕХАНИЗМ

```
главная G выполняет init A
↓ go worker()
другая G выполняется параллельно
главная G продолжает init B
```

ИНЖЕНЕРНЫЙ ВЫВОД

Последовательность вызовов `init` не запрещает конкурентную работу внутри них.

Опасность горутин до завершения инициализации пакетов

ЗАДАЧА И КОНТЕКСТ

Зависимости текущего пакета уже инициализированы, но некоторые другие пакеты программы могут получить свои значения позже.

МЕХАНИЗМ

```
init A → go worker()
↓ параллельный доступ к данным
пакет B еще не инициализирован
↓ риск скрытой гонки
```

ИНЖЕНЕРНЫЙ ВЫВОД

Не строим неявные зависимости между горутинами из `init` и будущими глобальными данными.

Почему запуск горутин из `init()` усложняет жизненный цикл

ЗАДАЧА И КОНТЕКСТ

Запуск долгоживущей горутины в `init()` допустим, но усложняет остановку, готовность и владение ресурсами.

МЕХАНИЗМ

```
init() создал worker
↓ кто остановит его?
↓ кто дождется готовности?
↓ с какими данными он работает?
```

ИНЖЕНЕРНЫЙ ВЫВОД

Для backend-сервисов часто проще явно запускать рабочие горутины из `main`.

Долгий `init()` откладывает `main.main`

GO / КОМАНДА

```
func init() {
    time.Sleep(3 * time.Second)
}
```

ЧТО ПРОИСХОДИТ

Процесс существует, планировщик работает, но пользовательский `main` еще не вызван. Готовность сервиса задерживается.

ИНЖЕНЕРНЫЙ ВЫВОД

Задержка запуска появляется раньше первой строки `main`.

Особенно опасные операции внутри `init()`

ЗАДАЧА И КОНТЕКСТ

Сеть, БД, удаленная конфигурация, повторы и тяжелый прогрев в `init()` откладывают `main.main`.

МЕХАНИЗМ

```
сетевой запрос / БД
повторные попытки
распаковка / прогрев кеша
↓ зависимость старта от внешнего мира
```

ИНЖЕНЕРНЫЙ ВЫВОД

Критические внешние операции лучше запускать с явными сроками и обработкой ошибок.

Почему `init()` неудобен для управляемого запуска

ЗАДАЧА И КОНТЕКСТ

`init()` вызывается автоматически, не принимает аргументов и не возвращает ошибку.

МЕХАНИЗМ

```
нет context и таймаута параметром
нет возвращаемого error
↓ сложнее управлять отказом и отменой
```

ИНЖЕНЕРНЫЙ ВЫВОД

Явная стартовая функция дает контроль над зависимостями и ошибками.

Как сообщить об ошибке из `init()`?

ЗАДАЧА И КОНТЕКСТ

Когда инициализация завершается с ошибкой, `init()` не может вернуть ее вызывающему коду.

МЕХАНИЗМ

```
внутри init: ошибка
↓ скрыть / panic / записать сбойное
состояние
vs
func run() error
```

ИНЖЕНЕРНЫЙ ВЫВОД

Для внешних зависимостей предпочтительнее управляемая инициализация.

`inittrace` показывает стоимость пакетов

ОПОРНАЯ МОДЕЛЬ

`GODEBUG=inittrace=1` включает сбор времени и аллокаций во время инициализации.

АЛГОРИТМ

```
пакет A: время / байты / аллокации
↓
пакет B: время / байты / аллокации
↓
найти дорогую инициализацию
```

ИНЖЕНЕРНЫЙ ВЫВОД

Это диагностика `init`, не измерение всего ELF-загрузчика.

Почему `inittrace` не объяснит время до рантайма

ЗАДАЧА И КОНТЕКСТ

`inittrace` начинается только в `runtime.main` и измеряет задачи инициализации, а не весь запуск процесса.

МЕХАНИЗМ

```
загрузчик / _rt0_* / schedinit
↓ до inittrace
задачи пакетов → inittrace
↓ main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Отсутствие затрат в `inittrace` не исключает задержку на более раннем слое.

Почему измерение `init` отключается после запуска

ЗАДАЧА И КОНТЕКСТ

После завершения инициализации рантайм отключает сбор статистики `init`, чтобы не платить за него всю жизнь процесса.

МЕХАНИЗМ

```
runtime.main запускает inittrace
↓ выполнить inittasks
↓ inittrace.active = false
```

ИНЖЕНЕРНЫЙ ВЫВОД

Диагностическая работа ограничена стадией инициализации пакетов.

Рантайм отмечает завершение `init`

GO / КОМАНДА

```
mainInitDone.Store(true)
close(mainInitDoneChan)
```

ЧТО ПРОИСХОДИТ

Так другие части рантайма и cgo могут дождаться окончания инициализации.

ИНЖЕНЕРНЫЙ ВЫВОД

Завершение инициализации имеет явную точку синхронизации.

Зачем нужен `mainInitDoneChan`

ЗАДАЧА И КОНТЕКСТ

`mainInitDoneChan` позволяет другим частям рантайма, включая cgo, дождаться корректного завершения инициализации.

МЕХАНИЗМ

```
mainInitDone еще false
↓ ожидание по mainInitDoneChan
↓ закрытие канала после init
обычное взаимодействие разрешено
```

ИНЖЕНЕРНЫЙ ВЫВОД

Сигнал защищает рантайм от слишком ранних внешних вызовов.

После `init` главная G освобождает поток ОС

ЗАДАЧА И КОНТЕКСТ

После инициализации `runtime.main` снимает свою привязку к стартовому потоку ОС, если пользователь ее не продлил.

МЕХАНИЗМ

```
init завершен
↓ unlockOSThread()
↓ обычное планирование главной G
переход между M возможен
```

ИНЖЕНЕРНЫЙ ВЫВОД

Специальная привязка нужна на время `init`, а не на всю жизнь программы.

Зачем освобождать поток именно после `init`

ЗАДАЧА И КОНТЕКСТ

Гарантия выполнения `init` на стартовом потоке требует удерживать главную G до конца задач инициализации.

МЕХАНИЗМ

```
lockOSThread() перед init
↓ все пакеты инициализировались
↓ unlockOSThread()
↓ main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Освобождение потока раньше нарушило бы гарантию выполнения `init`.

Только теперь пользовательский `main.main`

ОПОРНАЯ МОДЕЛЬ

Все пакеты инициализированы, основа рантайма работает, стартовый поток может быть освобожден.

АЛГОРИТМ

```
runtime.main
↓ doInit(runtime_inittasks)
↓ gcenable
↓ doInit(m.inittasks)
↓ mainInitDone
↓ fn := main_main; fn()
```

ИНЖЕНЕРНЫЙ ВЫВОД

`main.main` — последняя стадия запуска, а не первая инструкция процесса.

Почему переход в `main.main` косвенный

ЗАДАЧА И КОНТЕКСТ

Рантайм и пользовательский пакет компонуются независимо, поэтому переход в `main.main` выполняется через связанную компоновщиком ссылку.

МЕХАНИЗМ

```
runtime.main
↓ fn := main_main
↓ fn()
переход в пользовательский main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Последняя стрелка запуска тоже связана с работой компоновщика.

Что готово к первой строке `main.main`

ЗАДАЧА И КОНТЕКСТ

К первой строке `main.main` уже готовы G/M/P, аллокатор, GC, `sysmon` и метаданные типов.

МЕХАНИЗМ

```
планировщик и P работают
GC включен / аллокатор готов
глобальные переменные и init завершены
↓ main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Пользовательский `main` начинает работу внутри полностью подготовленного рантайма.

`main.main` — поздний этап жизни Go-процесса

ОПОРНАЯ МОДЕЛЬ

Пользователь видит функцию `main()` как начало программы, но к ней приходит уже работающий рантайм.

АЛГОРИТМ

```
загрузчик ОС → rt0
↓ schedinit → procresize
↓ mstart → runtime.main
↓ runtime_inittasks
↓ инициализация пакетов
main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

К моменту `main.main` уже готовы планировщик, аллокатор, GC и глобальные данные.

Не нужно запоминать 50 вызовов

Три границы готовности позволяют объяснять порядок без заучивания всех функций.

A — создание основы
B — запуск планировщика
C — инициализация приложения

Делим длинный запуск на три понятные фазы

ЗАДАЧА И КОНТЕКСТ

Порядок запуска проще понимать как три этапа, а не длинный список частных функций.

МЕХАНИЗМ

```
A — создать m0/g0 и основу
B — создать P и запустить G
C — выполнить init и вызвать main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Эти этапы имеют разные условия готовности и разные ошибки.

Фаза A — создать основу рантайма

ОПОРНАЯ МОДЕЛЬ

Первый поток ОС получает представление `m0/g0`; затем поднимаются ОС-зависимые и фундаментальные подсистемы.

АЛГОРИТМ

```
точка входа
↓ rt0_go
↓ исходный стек → g0
↓ m0 ↔ g0 / TLS
↓ args → osinit → schedinit
```

ИНЖЕНЕРНЫЙ ВЫВОД

Фаза A использует ресурсы ОС, не обычную работу планировщика.

Фаза B — начать исполнение обычных G

ОПОРНАЯ МОДЕЛЬ

Набор P уже существует; теперь планировщик получает первую работу.

АЛГОРИТМ

```
procresize → P
↓ newproc(runtime.main)
↓ mstart → schedule
↓ G(runtime.main)
```

ИНЖЕНЕРНЫЙ ВЫВОД

Фаза B переводит подготовленную основу в рабочий планировщик.

Фаза C — подготовить приложение

ОПОРНАЯ МОДЕЛЬ

`runtime.main` запускает системные фоновые механизмы и выполняет задачи инициализации.

АЛГОРИТМ

```
runtime.main → sysmon
↓ runtime_inittasks
↓ gcenable
↓ m.inittasks
↓ mainInitDone → main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Фаза C завершается первым вызовом пользовательского `main`.

От мира ОС к работающему приложению

ЗАДАЧА И КОНТЕКСТ

Каждый уровень использует подготовленные ресурсы предыдущего: ОС рантайм планировщик приложение.

МЕХАНИЗМ

```
ОС дала поток и стек
↓ рантайм создал свои структуры
↓ планировщик запустил G
↓ приложение прошло init
```

ИНЖЕНЕРНЫЙ ВЫВОД

Причинность идет снизу вверх: нельзя использовать подсистему до ее подготовки.

Когда какая подсистема готова?

МЕХАНИЗМ	НАЧАЛО RT0	ПОСЛЕ SCHEDINIT	RUNTIME.MAIN	ПЕРЕД MAIN
m0/g0	поднимаются	готовы	готовы	готовы
Подсистема стеков	нет	готова	готова	готова
Аллокатор	нет	готов	готов	готов
Карты и хеши	нет	готовы	готовы	готовы
Метаданные типов	нет	готовы	готовы	готовы
Состояние GC	нет	создано	включен	включен
Набор P	нет	готов	готов	готов
Планировщик	нет	еще не запущен	работает	работает
sysmon	нет	нет	запускается	работает
Инициализация пакетов	нет	нет	выполняется	закончена

ИНЖЕНЕРНЫЙ ВЫВОД

Таблица показывает границы готовности рантайма и момент появления основных механизмов.

Таблица готовности отвечает на практический вопрос

ЗАДАЧА И КОНТЕКСТ

Таблица готовности показывает, какие операции доступны на каждом этапе запуска.

МЕХАНИЗМ

```
в rt0_go — только начальные ресурсы
после schedinit — аллокатор и P
в runtime.main — планировщик и init
перед main.main — все готово
```

ИНЖЕНЕРНЫЙ ВЫВОД

Полезнее понимать границы готовности, чем заучивать отдельные вызовы.

Стоимость запуска под нагрузкой

Короткая инициализация одного процесса превращается в проблему при массовом запуске.

```
холодный запуск → readiness
всплески соединений и памяти
CPU-лимиты контейнера
```

Когда одноразовый запуск влияет на SLA

ЗАДАЧА И КОНТЕКСТ

Даже одноразовая работа может быть критичной для бессерверных функций, автоскейлинга, CLI и коротких заданий.

МЕХАНИЗМ

```
долгоживущий сервис: старт один раз
массовый запуск / короткая задача
↓ время инициализации повторяется
↓ задержка становится SLA
```

ИНЖЕНЕРНЫЙ ВЫВОД

Цена первоначального запуска зависит от жизненного цикла приложения.

До готовности pod проходит несколько разных слоев

ОПОРНАЯ МОДЕЛЬ

Readiness — метрика приложения, а не факт существования процесса.

АЛГОРИТМ

```
планирование pod / запуск контейнера
↓ execve / загрузчик ОС
↓ rt0 → schedinit
↓ runtime.main → init
↓ main.main / listener
↓ readiness
```

ИНЖЕНЕРНЫЙ ВЫВОД

Проблему холодного запуска нужно локализовать по этапам.

Наличие PID не означает готовность приложения

ЗАДАЧА И КОНТЕКСТ

Создание процесса и готовность сервиса — разные события. После появления PID код может еще выполнять `init`.

МЕХАНИЗМ

```
процесс уже существует
но rt0 / schedinit / init продолжаются
↓ слушатель еще не готов
readiness должна дождаться приложения
```

ИНЖЕНЕРНЫЙ ВЫВОД

Проверка готовности должна отражать состояние сервиса, а не наличие процесса.

Массовый запуск усиливает цену `init`

ЗАДАЧА И КОНТЕКСТ

Небольшая цена `init()` одного процесса при запуске сотен реплик превращается в нагрузочный всплеск.

МЕХАНИЗМ

```
1 pod: 500 мс инициализации
1000 pod одновременно:
всплеск CPU / памяти / I/O
массовые подключения к БД
```

ИНЖЕНЕРНЫЙ ВЫВОД

Инициализация становится распределенной нагрузкой при массовом запуске.

Сетевые вызовы в `init` могут устроить всплеск обращений

ЗАДАЧА И КОНТЕКСТ

Если сотни реплик еще до `main` обращаются к БД и удаленной конфигурации, они создают одновременный всплеск запросов.

МЕХАНИЗМ

```
500 реплик
↓ каждый init() делает запрос к БД
↓ множество одновременных соединений
↓ рост задержек внешней системы
```

ИНЖЕНЕРНЫЙ ВЫВОД

Сетевые зависимости полезнее запускать явно и контролировать их параллелизм.

Время запуска измеряется несколькими отметками

ОПОРНАЯ МОДЕЛЬ

Отдельно наблюдаем стадии процесса, инициализации и готовности приложения.

АЛГОРИТМ

```
T0 — запуск процесса
T1 — старт кода рантайма
T2 — конец инициализации пакетов
T3 — вход в main.main
T4 — слушатель готов
T5 — readiness
```

ИНЖЕНЕРНЫЙ ВЫВОД

Одна цифра задержка запуска не показывает, какой слой задержался.

Что входит в стоимость `schedinit`

ЗАДАЧА И КОНТЕКСТ

`schedinit` обычно занимает мало времени, но включает настройку аллокатора, метаданных типов, GC и набора P.

МЕХАНИЗМ

```
стек / память / типы
↓ подготовка GC
↓ создать P
↓ перейти к runtime.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Не приписываем замедление `schedinit` без измерения конкретного этапа.

В серверных приложениях чаще тормозит инициализация приложения

ЗАДАЧА И КОНТЕКСТ

На практике чаще дорогими оказываются глобальные переменные, пользовательский `init` и действия с данными, чем сама настройка `m0/g0`.

МЕХАНИЗМ

```
глобальные инициализаторы
func init()
cgo / большие встроенные данные
↓ потенциальная задержка до main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Сначала измеряем содержимое инициализации приложения, а не оптимизируем рантайм.

Почему стоит понимать даже быстрый bootstrap

ЗАДАЧА И КОНТЕКСТ

Даже если начальная инициализация обычно быстрая, знание ее границ помогает читать ранние аварии и трассировки.

МЕХАНИЗМ

```
сбой до main
↓ загрузчик? rt0? schedinit?
↓ runtime.main? пакетный init?
↓ выбрать правильный инструмент
```

ИНЖЕНЕРНЫЙ ВЫВОД

Фазовая модель позволяет не путать разные источники раннего сбоя.

GOMAXPROCS и CPU-ограничения контейнера

ЗАДАЧА И КОНТЕКСТ

В контейнере доступное процессорное время может быть значительно меньше числа CPU хоста.

МЕХАНИЗМ

```
хост: много CPU
контейнер: привязка CPU / квота cgroup
↓ начальное GOMAXPROCS
↓ соответствующее число P
```

ИНЖЕНЕРНЫЙ ВЫВОД

Планировщик Go должен учитывать ограничения среды выполнения.

Почему не стоит ориентироваться на CPU всего хоста

ЗАДАЧА И КОНТЕКСТ

На хосте может быть 64 ядра, а квота контейнера близка к двум CPU. Создавать P только по хосту часто невыгодно.

МЕХАНИЗМ

```
64 CPU на хосте
↓ лимит контейнера ≈ 2 CPU
↓ выбор GOMAXPROCS
↓ меньше лишней конкуренции / throttling
```

ИНЖЕНЕРНЫЙ ВЫВОД

Доступный CPU контейнера не равен общему числу CPU машины.

Два P — не два физических ядра

ЗАДАЧА

`GOMAXPROCS=2` ограничивает число P, способных параллельно исполнять Go-код.

МЕХАНИЗМ

ОС и cgroup планируют реальные CPU отдельно; квота не резервирует два ядра.

ИНЖЕНЕРНЫЙ ВЫВОД

P — ресурс планировщика Go, а не выделенное ядро ОС.

Лимит CPU может измениться после запуска

ЗАДАЧА И КОНТЕКСТ

Текущий рантайм может периодически обновлять автоматически выбранное GOMAXPROCS, когда меняются лимиты или привязка CPU.

МЕХАНИЗМ

```
при старте → число P
↓ лимиты среды изменились
↓ повторная оценка политики
↓ возможна перестройка набора P
```

ИНЖЕНЕРНЫЙ ВЫВОД

Решение о числе P не всегда навсегда привязано к моменту запуска.

Где именно остановился запуск?

Сначала отделяем загрузчик ОС, рантайм и инициализацию пакетов.

```
strace / временные отметки
GODEBUG=inittrace=1
дампы горутин / профилирование
```

Сначала выясняем, где остановился запуск

ОПОРНАЯ МОДЕЛЬ

У `4.5` и `4.6` разные причины и инструменты диагностики.

АЛГОРИТМ

```
бинарник не запускается
→ загрузчик / библиотека / ISA

процесс есть, main нет
→ rt0 / runtime.main / init

main есть, readiness нет
→ инициализация приложения
```

ИНЖЕНЕРНЫЙ ВЫВОД

Проверка начинается с правильной границы, а не с случайного профиля.

Когда сбой происходит еще до Go-рантайма

ЗАДАЧА И КОНТЕКСТ

`exec format error`, отсутствующий загрузчик, библиотека или неподдерживаемая ISA указывают на стадию до нормальной работы Go.

МЕХАНИЗМ

```
файл / ELF / интерпретатор
↓ загрузчик ОС
↓ ранняя проверка CPU
еще до runtime.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Такие ошибки диагностируются на уровне ОС, формата файла и платформы.

Процесс есть, а пользовательских логов нет

ЗАДАЧА И КОНТЕКСТ

Наличие процесса без пользовательских логов может означать работу рантайма или пакетных инициализаторов.

МЕХАНИЗМ

```
PID уже есть
↓ rt0/schedinit
↓ runtime.main
↓ init()
позже → пользовательские логи
```

ИНЖЕНЕРНЫЙ ВЫВОД

Не делаем вывод о зависании по одному отсутствующему сообщению из `main`.

Первый инструмент для дорогого `init`

GO / КОМАНДА

```
GODEBUG=inittrace=1 ./app
```

ЧТО ПРОИСХОДИТ

По каждому пакету можно увидеть время и число аллокаций. Это помогает найти скрытый дорогостоящий инициализатор.

ИНЖЕНЕРНЫЙ ВЫВОД

Если время потеряно до `runtime.main`, потребуется другая диагностика.

Много аллокаций в `init` не означает утечку

ЗАДАЧА И КОНТЕКСТ

Аллокации при инициализации могут создавать полезные долгоживущие структуры. Большие показатели не обязательно означают утечку.

МЕХАНИЗМ

```
inittrace: много байт/аллокаций
↓ какие объекты создаются?
↓ живут ли они все время процесса?
↓ нужны ли они до первого запроса?
```

ИНЖЕНЕРНЫЙ ВЫВОД

Измеряем назначение и время жизни объектов, а не только их количество.

Что делать, если `inittrace` пуст

ЗАДАЧА И КОНТЕКСТ

Если `inittrace` не показывает затрат, задержка могла возникнуть раньше: в загрузчике ОС, cgo или рантайме до инициализации пакетов.

МЕХАНИЗМ

```
inittrace пуст
↓ strace / временные отметки
↓ диагностика загрузчика и rt0
↓ при необходимости ранняя трассировка
```

ИНЖЕНЕРНЫЙ ВЫВОД

Инструмент, включенный поздно, не видит предыдущие этапы.

`strace` помогает отделить загрузчик от Go

ЗАДАЧА И КОНТЕКСТ

`strace` помогает заметить открытие библиотек и вызовы загрузчика в пользовательском пространстве.

МЕХАНИЗМ

```
execve
↓ openat / mmap / mprotect
↓ библиотеки и загрузчик
потом → Go-рантайм
```

ИНЖЕНЕРНЫЙ ВЫВОД

Внутренние действия загрузчика ядра не обязаны быть отдельными системными вызовами.

Ранние временные отметки иногда полезнее профиля

ЗАДАЧА И КОНТЕКСТ

Для первоначальной локализации достаточно записать отметки начала раннего `init`, входа в `main` и готовности приложения.

МЕХАНИЗМ

```
T0: ранний init
T1: вход в main.main
T2: готовность listener
↓ сравнить интервалы
```

ИНЖЕНЕРНЫЙ ВЫВОД

Не всегда нужна дорогостоящая профилировка: сначала разбиваем время на этапы.

Как профилировать запуск до HTTP-pprof

ЗАДАЧА И КОНТЕКСТ

HTTP-обработчик pprof еще не работает, пока приложение не создало сервер. Тяжелый ранний CPU-код нужно измерять отдельно.

МЕХАНИЗМ

```
до HTTP: нет /debug/pprof
↓ включить профилирование программно
↓ завершить запись профиля
↓ оценить влияние измерения
```

ИНЖЕНЕРНЫЙ ВЫВОД

Инструментирование должно быть доступно на исследуемом этапе запуска.

Что покажет ранняя трассировка `runtime/trace`

ЗАДАЧА И КОНТЕКСТ

Трассировку `runtime/trace` можно начать из достаточно раннего кода приложения и увидеть горутины, блокировки и таймеры.

МЕХАНИЗМ

```
запустить trace в init / main
↓ увидеть работу планировщика
но: _rt0/schedinit уже прошли
```

ИНЖЕНЕРНЫЙ ВЫВОД

Обычная трассировка приложения не покрывает самые ранние инструкции Go.

Дамп горутин при зависшем `init()`

ЗАДАЧА И КОНТЕКСТ

Если процесс жив, но пакетный `init` застрял, по SIGQUIT часто можно получить дамп горутин.

МЕХАНИЗМ

```
SIGQUIT
↓ стеки горутин
↓ главная G в init
↓ найти место ожидания
```

ИНЖЕНЕРНЫЙ ВЫВОД

Дамп горутин помогает отличить зависший `init` от ошибок загрузчика.

Почему ранняя авария сложнее в диагностике

ЗАДАЧА И КОНТЕКСТ

При аварии до полноценного запуска планировщика обычные средства трассировки и аварийного дампа могут быть ограничены.

МЕХАНИЗМ

```
сбой до создания P
↓ не все подсистемы диагностики готовы
↓ специальные аварийные пути рантайма
```

ИНЖЕНЕРНЫЙ ВЫВОД

Ранние сбои требуют учитывать фактическую готовность самого рантайма.

Нарушение инварианта при `procresize`

ЗАДАЧА И КОНТЕКСТ

Во время `schedinit` рантайм строго проверяет инварианты: например, при `procresize` не должно появиться неожиданной готовой G.

МЕХАНИЗМ

```
мир еще остановлен
↓ procresize
неожиданная готовая G?
↓ внутренняя фатальная ошибка
```

ИНЖЕНЕРНЫЙ ВЫВОД

Первоначальный запуск имеет более жесткие ограничения, чем штатная работа.

Докажем все на работающем Go

Эксперименты от точки входа до `inittrace`, блокировок и CPU-лимитов.

```
исходник → запускаем
ломаем → наблюдаем
измеряем → объясняем
```

Проверяем точку входа исполняемого файла

GO / КОМАНДА

```
readelf -h ./app
go tool nm ./app
go tool objdump ./app
```

ЧТО ПРОИСХОДИТ

Найти `_rt0_amd64`, `runtime.rt0_go` и адрес входа, затем сравнить с `main.main`.

ИНЖЕНЕРНЫЙ ВЫВОД

Точка входа исполняемого файла не равна пользовательскому `main`.

Эксперимент 1: доказать, что вход не равен `main.main`

ЗАДАЧА И КОНТЕКСТ

Эксперимент должен восстановить путь от ELF entry к `runtime.rt0_go`, а не просто найти символы по имени.

МЕХАНИЗМ

```
readelf -h → entry
↓ objdump/nm
_rt0_amd64 → rt0_go
↓ сравнить с main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Физическая точка входа и пользовательская функция `main` различаются.

Эксперимент 2: разобрать `rt0_go` по цепочке

ЗАДАЧА И КОНТЕКСТ

Читаем `asm_amd64.s` выборочно: ищем подготовку стека, TLS, связь `m0/g0` и вызовы дальнейшей инициализации.

МЕХАНИЗМ

```
rt0_go: стек g0
↓ TLS / m0.g0 / g0.m
↓ args/osinit/schedinit
↓ newproc/mstart
```

ИНЖЕНЕРНЫЙ ВЫВОД

Чтение исходников привязываем к переходам состояния рантайма.

Исходники рантайма читаем по механизму, не подряд

ЗАДАЧА И КОНТЕКСТ

Линейное чтение тысяч строк исходника редко дает хорошую причинную модель. Начинаем с конкретного перехода.

МЕХАНИЗМ

```
вопрос о механизме
↓ найти точку входа
↓ проследить вызовы и изменения данных
↓ проверить условия перехода
```

ИНЖЕНЕРНЫЙ ВЫВОД

Исходники рантайма изучают как техническое расследование, а не как роман.

Статические объекты `m0/g0` в исходнике рантайма

GO / КОМАНДА

```
var (
    m0 m
    g0 g
)
```

ЧТО ПРОИСХОДИТ

Открыть `runtime/proc.go`; связать статическое выделение памяти с отображенными данными ELF.

ИНЖЕНЕРНЫЙ ВЫВОД

Первому M/G не нужен еще неработающий аллокатор.

Эксперимент 4: восстановить зависимости `schedinit`

ЗАДАЧА И КОНТЕКСТ

В исходнике `schedinit` нужно восстановить только основные зависимости, не заучивая все поля и вызовы.

МЕХАНИЗМ

```
стек → случайность → аллокатор
↓ CPU / алгоритмы карт
↓ M и метаданные типов
↓ GC → P
```

ИНЖЕНЕРНЫЙ ВЫВОД

Крупные подсистемы объясняют порядок значительно лучше списка мелких функций.

Порядок пакетов и стоимость через `inittrace`

GO / КОМАНДА

```
GODEBUG=inittrace=1 ./app
```

ЧТО ПРОИСХОДИТ

Сделать пакеты A B C с небольшими аллокациями в `init`. Сравнить последовательность, время и байты.

ИНЖЕНЕРНЫЙ ВЫВОД

Практика подтверждает порядок зависимостей и цену каждого `init`.

Что измерять при инициализации пакетов

ЗАДАЧА И КОНТЕКСТ

Для каждого пакета `inittrace` дает время, число аллокаций и выделенные байты.

МЕХАНИЗМ

```
пакет A → время / байты / аллокации
пакет B → время / байты / аллокации
↓ добавить time.Sleep
↓ сравнить результаты
```

ИНЖЕНЕРНЫЙ ВЫВОД

Практика доказывает измеримость стоимости инициализации.

Эксперимент 6: глобальный инициализатор

ЗАДАЧА И КОНТЕКСТ

Вынести работу из `func init()` в выражение инициализации глобальной переменной и повторить замер.

МЕХАНИЗМ

```
var x = expensive()
↓ инициализация пакета
↓ inittrace
↓ цена сохраняется
```

ИНЖЕНЕРНЫЙ ВЫВОД

Глобальная переменная может запускать тяжелый код до `main`.

Горутина из `init()`

GO / КОМАНДА

```
func init() {
    go background()
}
```

ЧТО ПРОИСХОДИТ

Добавить синхронизацию для воспроизводимости. Убедиться, что планировщик способен выполнять G до `main.main`.

ИНЖЕНЕРНЫЙ ВЫВОД

Пример специально показывает запуск конкурентной работы во время инициализации.

Намеренная блокировка `init()`

GO / КОМАНДА

```
func init() {
    select {}
}
```

ЧТО ПРОИСХОДИТ

ВНИМАНИЕ: учебный дефект. Главная G не завершает инициализацию; исследуем состояние процесса и дамп горутин.

ИНЖЕНЕРНЫЙ ВЫВОД

Нельзя рассматривать заблокированный `init` как отсутствие созданного процесса.

Эксперимент 9: `LockOSThread`

ЗАДАЧА И КОНТЕКСТ

В `init()` вызвать `runtime.LockOSThread()` и сравнить идентификатор потока ОС при инициализации и в `main`.

МЕХАНИЗМ

```
init: LockOSThread()
↓ запомнить TID
main: получить TID
↓ сравнить
```

ИНЖЕНЕРНЫЙ ВЫВОД

Эксперимент проверяет привязку главной G к стартовому потоку ОС.

Сравнить исходное GOMAXPROCS

GO / КОМАНДА

```
GOMAXPROCS=1 ./app
GOMAXPROCS=4 ./app
```

ЧТО ПРОИСХОДИТ

Приложение выводит `runtime.GOMAXPROCS(0)`.
Связываем окружение с `schedinit → procresize`.

ИНЖЕНЕРНЫЙ ВЫВОД

Эксперимент доказывает начальный выбор числа P.

Эксперимент 11: лимиты CPU контейнера

ЗАДАЧА И КОНТЕКСТ

Запустить приложение в контейнере с заданным лимитом CPU и прочитать число доступных CPU и P.

МЕХАНИЗМ

```
CPU limit контейнера
↓ runtime.NumCPU()
↓ runtime.GOMAXPROCS(0)
↓ сопоставить с версией Go
```

ИНЖЕНЕРНЫЙ ВЫВОД

Для наблюдения политики важно фиксировать и версию инструментария, и лимиты контейнера.

Эксперимент 12: много процессов с тяжелым `init`

ЗАДАЧА И КОНТЕКСТ

Сделать инициализатор с большой аллокацией и запустить много процессов параллельно.

МЕХАНИЗМ

```
один процесс: выделить большой объект
↓ N одновременных запусков
измерить время / CPU / память
↓ перенести работу в контролируемый прогрев
```

ИНЖЕНЕРНЫЙ ВЫВОД

Всплеск запуска может создать давление на память и CPU вне обработки запросов.

Сеть в `init()` — намеренно плохой пример

ЗАДАЧА

Инициализатор обращается к локальному сервису, который специально отвечает с задержкой.

МЕХАНИЗМ

Перенести запрос в явную стартовую функцию с `context` и таймаутом.

ИНЖЕНЕРНЫЙ ВЫВОД

Зависимости приложения должны запускаться контролируемо, с обработкой ошибок и сроками.

Эксперимент 14: загрузчик против `init`

ЗАДАЧА И КОНТЕКСТ

Собрать динамический бинарник с тяжелым `init()` и сопоставить системные вызовы загрузчика со временем пакетной инициализации.

МЕХАНИЗМ

```
strace → загрузчик / библиотеки
↓ inittrace → пакеты
↓ отметка входа в main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Системная трассировка и измерение Go показывают разные участки запуска.

Эксперимент 15: `runtime.main main.main`

ЗАДАЧА И КОНТЕКСТ

В готовом бинарнике найти `runtime.main` и `main.main`, затем посмотреть переход между ними.

МЕХАНИЗМ

```
go tool nm → оба символа
↓ go tool objdump
↓ косвенный вызов в runtime.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Последний вызов происходит после завершения инициализации пакетов.

Спецификация, API и детали реализации

Разделяем, что Go гарантирует, и как Go 1.27.x устроен внутри.

```
семантика init
API GOMAXPROCS
внутренние rt0/schedinit/procresize
```

Спецификация гарантирует порядок инициализации

ЗАДАЧА

Сначала зависимости, затем переменные пакета и его `init()`; до `main.main` инициализация завершается.

МЕХАНИЗМ

Внутренние `rt0`, `schedinit` и `mstart` — реализация, а не обещание языка.

ИНЖЕНЕРНЫЙ ВЫВОД

Наблюдаемое поведение отделяем от конкретного устройства Go 1.27.x.

Что гарантирует публичный API рантайма

ЗАДАЧА И КОНТЕКСТ

Документированное поведение `LockOSThread` и API `GOMAXPROCS` можно использовать в приложении.

МЕХАНИЗМ

```
публичный API: контракты стабильны
частные функции рантайма: детали версии
↓ применять на соответствующем уровне
```

ИНЖЕНЕРНЫЙ ВЫВОД

Нельзя подменять гарантию API предположением о внутреннем порядке вызовов.

Что относится к деталям версии Go

ОПОРНАЯ МОДЕЛЬ

Полезно понимать реализацию, но нельзя завязывать корректность приложения на ее частные функции.

АЛГОРИТМ

```
_rt0_* → rt0_go
m0/g0 и TLS
osinit/schedinit/procresize
mstart0/mstart1
runtime.main и mainInitDone
```

ИНЖЕНЕРНЫЙ ВЫВОД

Эта последовательность может измениться между версиями рантайма.

Почему нельзя зависеть от частных функций рантайма

ЗАДАЧА И КОНТЕКСТ

Детали `m0/g0`, TLS и `schedinit` помогают понимать Go 1.27.x, но не являются частью спецификации языка.

МЕХАНИЗМ

```
приложение опирается на:
семантику языка + публичные API
а не на runtime.main
по конкретной строке исходника
```

ИНЖЕНЕРНЫЙ ВЫВОД

Корректность приложения нельзя связывать с частными функциями рантайма.

GOMAXPROCS: поведение зависит от версии Go

ЗАДАЧА И КОНТЕКСТ

Решение о GOMAXPROCS по умолчанию и его возможном обновлении менялось между версиями Go.

МЕХАНИЗМ

```
версия Go
+ GOMAXPROCS в окружении
+ cgroup/привязка
↓ фактическое число P
```

ИНЖЕНЕРНЫЙ ВЫВОД

Советы по CPU-лимитам без указания версии Go могут быть неверными.

Миф: ОС вызывает `main.main`

УПРОЩЕНИЕ

ОС передает управление точке входа исполняемого файла. Это еще не пользовательская функция.

ТОЧНАЯ МОДЕЛЬ

```
_rt0_* runtime.rt0_go
runtime.main init
main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

До `main.main` рантайм уже поднял планировщик, сборщик мусора и инициализировал пакеты.

Миф: Планировщик создает первый M

УПРОЩЕНИЕ

Начальный поток ОС уже создан загрузчиком. Планировщик еще не существует.

ТОЧНАЯ МОДЕЛЬ

первый поток ОС статический `m0`
`m0` связывается с `g0`
позже `schedinit`

ИНЖЕНЕРНЫЙ ВЫВОД

Первый M появляется благодаря ресурсам ОС; механизм создания следующих M работает позже.

Миф: Первый `g0` создается как обычная горутина

УПРОЩЕНИЕ

`g0` находится в статических данных исполняемого файла. Ее стек — уже существующий стек потока ОС.

ТОЧНАЯ МОДЕЛЬ

статический `g0`
стек первого потока ОС
`stack.lo / stack.hi`
готов системный стек

ИНЖЕНЕРНЫЙ ВЫВОД

Пока аллокатор не запущен, обычным способом создать G со своим стеком нельзя.

Миф: Аллокатор нужен, чтобы создать `m0/g0`

УПРОЩЕНИЕ

`m0` и `g0` доступны раньше аллокатора: место для них подготовлено еще компоновщиком.

ТОЧНАЯ МОДЕЛЬ

данные executable `m0/g0`
начальный код рантайма
`schedinit`
`mallocinit`

ИНЖЕНЕРНЫЙ ВЫВОД

Статическая память разрывает зависимость «для запуска аллокатора уже нужен аллокатор».

Миф: P существует сразу после точки входа

УПРОЩЕНИЕ

На входе в процесс есть поток ОС, но набор P еще не подготовлен.

ТОЧНАЯ МОДЕЛЬ

`rt0_go` `osinit`
`schedinit`
начальное `GOMAXPROCS`
`procresize` создать P

ИНЖЕНЕРНЫЙ ВЫВОД

Право выполнять обычный Go-код появляется после подготовки P.

Миф: После `schedinit` уже выполняется `main`

УПРОЩЕНИЕ

После `schedinit` рантайм способен планировать G, но первая обычная G еще не была запущена.

ТОЧНАЯ МОДЕЛЬ

```
newproc(runtime.main)
 mstart
 schedule
 G(runtime.main)
```

ИНЖЕНЕРНЫЙ ВЫВОД

Готовность подсистем и фактическое исполнение первой G — разные рубежи.

Миф: GC включается одним шагом

УПРОЩЕНИЕ

Сначала `gcinit` подготавливает состояние сборщика, а `gcenable` включается позднее.

ТОЧНАЯ МОДЕЛЬ

`schedinit gcinit`
P и планировщик готовы
`runtime.main gcenable`

ИНЖЕНЕРНЫЙ ВЫВОД

Фоновая работа GC возможна только после подготовки зависимых подсистем.

Миф: Инициализация пакетов идет до планировщика

УПРОЩЕНИЕ

К моменту выполнения `init()` планировщик уже запустил обычную G с `runtime.main`.

ТОЧНАЯ МОДЕЛЬ

```
mstart → schedule
 G(runtime.main)
 doInit(m.inittasks)
```

ИНЖЕНЕРНЫЙ ВЫВОД

Код `init()` может создавать горутины, таймеры, каналы и другие обычные объекты Go.

Миф: До `main` других горутин быть не может

УПРОЩЕНИЕ

Сам список `init` выполняет главная G. Однако отдельный `init()` вправе запустить другие горутины.

ТОЧНАЯ МОДЕЛЬ

```
runtime.main init A
 go worker()
```

другая G уже выполняется
главная G продолжает `init`

ИНЖЕНЕРНЫЙ ВЫВОД

Последовательная инициализация пакетов не запрещает конкурентный код внутри `init()`.

Миф: Скрытая работа до `main` есть только в `init()`

УПРОЩЕНИЕ

Глобальные переменные могут вызывать функции при инициализации пакета.

ТОЧНАЯ МОДЕЛЬ

`var client = makeClient()`
вычисление до `main`
затем `init()`
`main.main`

ИНЖЕНЕРНЫЙ ВЫВОД

При медленном запуске проверяем глобальные инициализаторы наравне с `func init()`.

Миф: Если процесс существует, приложение уже запущено

УПРОЩЕНИЕ

PID появляется раньше загрузки всех подсистем и готовности сервиса принимать запросы.

ТОЧНАЯ МОДЕЛЬ

PID создан
загрузчик ОС
`schedinit` / `init`
`main.main` / сервер
readiness

ИНЖЕНЕРНЫЙ ВЫВОД

Проверка готовности должна отражать готовность приложения, а не только наличие процесса.

Итоговые связи: 1–5 из 28

МОДЕЛЬ

1. Go стартует на уже существующем потоке ОС, а не на обычной горутине.\n\n2. `_rt0_*` адаптирует точку входа к `runtime.rt0_go`.\n\n3. Статические `m0/g0` существуют еще до запуска аллокатора.

ИНЖЕНЕРНЫЕ СЛЕДСТВИЯ

4. Начальный стек потока ОС становится системным стеком `g0`.\n\n5. Перед вызовами рантайма надо настроить текущую G: TLS и регистры зависят от платформы.

ИНЖЕНЕРНЫЙ ВЫВОД

Это последовательность появления механизмов, а не набор независимых терминов.

Итоговые связи: 6–10 из 28

МОДЕЛЬ

6. Связи `m0.g0` и `g0.m` связывают поток ОС с сущностями Go.\n\n7. `runtime.args → osinit → schedinit` выполняются до первой обычной G.\n\n8. `schedinit` поднимает стек, случайные значения, аллокатор, метаданные, GC и P.

ИНЖЕНЕРНЫЕ СЛЕДСТВИЯ

9. `maps.AlgInit` демонстрирует, что даже `map` требует ранней подготовки.\n\n10. `gcinit` готовит GC, а `gcenable` позднее разрешает фоновую работу.

ИНЖЕНЕРНЫЙ ВЫВОД

Это последовательность появления механизмов, а не набор независимых терминов.

Итоговые связи: 11–15 из 28

МОДЕЛЬ

11. Начальное `GOMAXPROCS` учитывает настройки запуска; на Linux важны лимиты CPU.\n\n12. `procresize` создает набор P и подготавливает возможность выполнения Go-кода.\n\n13. Первая обычная G выполняет `runtime.main`, а не пользовательский `main.main`.

ИНЖЕНЕРНЫЕ СЛЕДСТВИЯ

14. `mstart → mstart0 → mstart1 → schedule` запускает обычный цикл планирования.\n\n15. После входа в `schedule` действует уже изученная модель G/M/P.

ИНЖЕНЕРНЫЙ ВЫВОД

Это последовательность появления механизмов, а не набор независимых терминов.

Итоговые связи: 16–20 из 28

МОДЕЛЬ

16. `runtime.main` запускает `sysmon` и удерживает стартовый поток на время `init`.\n\n17. Компоновщик заранее строит список задач `init`; рантайм выполняет их по порядку.\n\n18. Инициализация пакета включает глобальные переменные и функции `init()`.

ИНЖЕНЕРНЫЕ СЛЕДСТВИЯ

19. Главная G выполняет последовательность `init`, но эти функции могут запустить другие G.\n\n20. Тяжелая работа в `init()` увеличивает время до первой строки `main.main`.

ИНЖЕНЕРНЫЙ ВЫВОД

Это последовательность появления механизмов, а не набор независимых терминов.

Итоговые связи: 21–25 из 28

МОДЕЛЬ

21. `GODEBUG=inittrace=1` измеряет время и аллокации инициализации пакетов.\n\n22. `mainInitDone` сигнализирует об окончании `init` для рантайма и cgo.\n\n23. После инициализации главная G освобождает стартовый поток при отсутствии других блокировок.

ИНЖЕНЕРНЫЕ СЛЕДСТВИЯ

24. Вызов `main.main` следует только после настройки рантайма и всех пакетов.\n\n25. `main.main` — поздний рубеж запуска процесса, а не первая его инструкция.

ИНЖЕНЕРНЫЙ ВЫВОД

Это последовательность появления механизмов, а не набор независимых терминов.

Итоговые связи: 26–28 из 28

МОДЕЛЬ

26. Отдельно диагностируем загрузчик ОС, bootstrap рантайма и инициализацию приложения.\n\n27. Цена запуска важна для развертываний, автомасштабирования и коротких заданий.\n\n28. Точная последовательность частных функций зависит от версии Go, а не гарантируется языком.

ИНЖЕНЕРНЫЕ СЛЕДСТВИЯ

ИНЖЕНЕРНЫЙ ВЫВОД

Это последовательность появления механизмов, а не набор независимых терминов.

Финальная схема 1/3: от ОС до m0/g0

ПОСЛЕДОВАТЕЛЬНОСТЬ

```
загрузчик ОС
↓ точка входа _rt0_*
runtime.rt0_go
↓ исходный стек ОС
g0 + TLS / текущая G
↓
m0.g0 = &g0 / g0.m = &m0
```

ИНЖЕНЕРНЫЙ ВЫВОД

Первый рабочий контекст рантайма создается из ресурсов, уже предоставленных ОС.

Финальная схема 2/3: от `schedinit` до P

ПОСЛЕДОВАТЕЛЬНОСТЬ

```
runtime.args → runtime.osinit
↓ runtime.schedinit
стек / аллокатор / типы / GC
↓ начальное GOMAXPROCS
procresize
↓ набор P готов
```

ИНЖЕНЕРНЫЙ ВЫВОД

Аллокатор и GC настраиваются до первого обычного запуска планировщика.

Финальная схема 3/3: от первой G до `main.main`

ПОСЛЕДОВАТЕЛЬНОСТЬ

```
newproc(runtime.main)
↓ mstart → schedule
G(runtime.main)
↓ sysmon / задачи init
gcenable
↓ mainInitDone
main.main
```

ИНЖЕНЕРНЫЙ ВЫВОД

Пользовательский `main` получает полностью подготовленный рантайм.

От точки входа к работающему Go

Теперь видно, откуда берутся G/M/P, аллокатор, GC и инициализация пакетов.

```
ОС → _rt0_* → m0/g0
schedinit → P → runtime.main
init → main.main
```